

UCS503P - Software Engineering Project

MID-TERM REPORT

FoodLink

A Coordination Platform for Surplus Food Redistribution

Submitted by:

1024030909 Rishi Garg

1024030906 Aayushi Sud

1024031166 Himanshi Singla

1024030956 Yugam Heer

rgarg8.be24@thapar.edu asud.be24@thapar.edu

hsingla1.be24@thapar.edu yheer.be24@thapar.edu

Group: 3C64 BE Third Year, CSE

Submitted to:

Reaya Grewal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	9
1.1	Background and Context	9
1.1.1	Problem Statement	10
1.1.2	Motivation	10
1.1.3	Existing System	11
1.1.4	Proposed System	11
1.1.5	Scope	13
1.2	Project Objectives	14
1.3	Organisation of the Report	14
2	Methodology and Design	17
2.1	System Architecture	17
2.1.1	High-Level Design	17
2.1.2	Technical Stack	19
2.1.3	System Requirements	19
2.1.4	Functional Requirements	20
2.1.5	Non-Functional Requirements	20
2.1.6	Database Design	20
2.1.7	API and Backend Design	25
2.1.8	Frontend Architecture	27
2.2	Implementation Details	28
2.2.1	Module 1: Core Functionality — Donation Lifecycle	28
2.2.2	Module 2: Core Functionality — Recipient Matching and Explainability	32
2.2.3	Module 3: Core Functionality — Authentication, Authorization and Trust	36
2.2.4	Module 4: Secondary Features — Standing Requirements	38
2.2.5	Module 5: Secondary Features — Administration and Platform Metrics	39
2.2.6	Module 6: Secondary Features — Role Portals and User Workflows	40
2.2.7	Security and Privacy Considerations	41
3	Results and Evaluation	43
3.1	Testing Methodology	43
3.2	Performance Metrics	45
3.3	Analysis	45
3.3.1	Current Limitations	46

4	Feasibility Report	49
4.1	Technical Feasibility	49
4.1.1	Technology Stack	49
4.1.2	Frontend	49
4.1.3	Backend and APIs	50
4.1.4	Database	50
4.1.5	Authentication	50
4.1.6	Deployment Considerations	50
4.1.7	Implementation Complexity	51
4.2	Operational Feasibility	51
4.3	Economic Feasibility	53
4.4	Schedule Feasibility	54
4.5	Legal, Security and Privacy Feasibility	55
4.6	Feasibility Summary	56
5	Conclusion	59
5.1	Summary of Work	59
5.2	Key Findings	59
5.3	Future Work and Recommendations	60
5.4	Final Remarks	61
6	System Design: UML Diagrams	63
6.1	Use Case Model	63
6.1.1	Use Case Diagram	63
6.1.2	Use Case Scenarios	65
6.2	Class Diagram	80
6.3	Activity Diagram	83
	Bibliography	87

List of Figures

6.1	Use case diagram of FoodLink	64
6.2	Class diagram of the FoodLink domain model and matching module .	81
6.3	Activity diagram of the donation lifecycle	84

List of Tables

1.1	Informal coordination compared with the implemented FoodLink system	12
1.2	Project objectives, measurable criteria and verified mid-term status .	15
2.1	Architectural tiers of FoodLink and where each is implemented	17
2.2	Technology stack with the versions declared in the repository	20
2.3	Functional requirements FR-01 to FR-11 and implementation status .	21
2.4	Functional requirements FR-12 to FR-22 and implementation status .	22
2.5	Non-functional requirements, mechanisms and known gaps	23
2.6	Database tables, purpose and relationships	24
2.7	REST API routes, part 1: authentication, donations and organisations	26
2.8	REST API routes, part 2: metrics, administration and health	27
2.9	Donation lifecycle states	29
2.10	Lifecycle transitions, permitted roles and enforced conditions	30
2.11	Published weights of the five matching criteria (sum 1.00)	34
2.12	Threshold rules that generate the plain-language match explanation .	35
2.13	Matcher output for a 60-meal donation (computed with <code>score_pair</code>)	35
2.14	Read and disclosure scopes by role, as implemented	38
2.15	Web portals, their routes and the workflows they support	40
2.16	Security and privacy controls and residual risks	42
3.1	Backend test suite organised by the property under test	44
3.2	Validation results at mid-term	45
3.3	Evaluation metrics from the proposal and their mid-term status . . .	46
3.4	Principal challenges found by audit and how they were resolved . . .	47
4.1	Implementation maturity and remaining work by area	54
4.2	Summary of feasibility by dimension	57
6.1	Use case scenario UC-01: Register Account	66
6.2	Use case scenario UC-02: Sign In	67
6.3	Use case scenario UC-03: Post Donation	68
6.4	Use case scenario UC-04: View Ranked Matches and Match Explanation	69
6.5	Use case scenario UC-05: Accept Donation	70
6.6	Use case scenario UC-06: Claim Pickup	71
6.7	Use case scenario UC-07: Record Pickup and Delivery	72
6.8	Use case scenario UC-08: Confirm Receipt	73
6.9	Use case scenario UC-09: Cancel Donation (API only)	74
6.10	Use case scenario UC-10: Release Pickup (API only)	75

6.11 Use case scenario UC-11: Manage Standing Requirements and Browse Needs Board	76
6.12 Use case scenario UC-12: Edit Organisation Profile (verification-sensitive)	77
6.13 Use case scenario UC-13: Verify or Revoke Organisation	78
6.14 Use case scenario UC-14: Run Expiry Sweep	79
6.15 Use case scenario UC-15: Manage User Accounts (API only)	80
6.16 Summary of the remaining use cases	82

Introduction

1.1 Background and Context

Institutional kitchens routinely finish a service with edible food left over. College messes, hostel canteens, bakeries and banquet halls are typical sources. At the same time, shelters, community kitchens and non-governmental organisations (NGOs) a few kilometres away are short of meals. Food waste is a recognised problem at every scale [Uni24]. For a single campus or a tier-2/3 Indian city, however, the binding constraint is rarely the *supply* of surplus food or the *existence* of organisations willing to serve it. It is *coordination under a deadline*. Cooked food is safe to serve for only a few hours, and the phone calls and informal messages that currently connect a donor to a recipient often outlast that window.

FoodLink is the software system this project builds to close that gap. It is a web platform that brings four kinds of participant into one shared, tracked workflow:

- a **donor**, who posts surplus food with a pickup location and an absolute pickup deadline;
- a **recipient organisation** (an NGO or community kitchen), which is verified by an administrator before it can take custody of food;
- a **volunteer courier**, who collects the food and delivers it;
- a **platform administrator**, who vouches for organisations, maintains the platform and can act on behalf of any party.

The project is being developed for the course UCS503P (Software Engineering Project), 2026–27 odd semester, at Thapar Institute of Engineering and Technology. The course asks for three reports: a proposal in week 4, a prototype-stage progress report before the mid-semester examinations in week 7, and a final report in week 17 [Ven26]. This document is the mid-term (prototype-stage) report. The proposal committed to a web-based, workflow-centred platform with a transparent ranking heuristic [GSHS26]. This report describes the system that has actually been built against that proposal, how it was verified, and what remains.

A note on terminology. The repository is named *FoodBridge-AI*, and the product was at one point labelled “FoodLink AI”. FoodLink contains **no machine learning, deep learning or computer vision**. Recipient organisations are ranked by a transparent weighted sum of five published criteria, which a reader can recompute by hand

(Section 2.2.2). This is a deliberate design decision recorded in the project’s decision log as D-05 [Foo26a]. Every technical statement in this report was checked against the source code at commit `e588b35` [Foo26b]. Where the code and the project’s own documentation disagree, the code was followed.

1.1.1 Problem Statement

Surplus food is lost less for lack of food or willing recipients than for lack of a reliable, timely and trustworthy way to connect them. Five concrete problems follow from informal coordination:

1. **A hard perishability deadline.** Food has a window after which it must be discarded. Coordination that is not deadline-aware routinely misses it.
2. **Fragmentation.** A donor contacts whichever organisation it happens to know. There is no shared pool, so availability is invisible to everyone else, and the nearest suitable organisation may never hear of the food.
3. **No accountability or evidence.** Nobody can say afterwards how long it took for food to be claimed, whether it was actually delivered, or how much was lost. Without records, the process cannot be measured or improved.
4. **Last-mile coordination.** Food must be physically moved. Volunteer availability is informal and untracked, so a willing donor and a willing recipient can still fail to complete a handover.
5. **Trust and privacy.** A donor handing food to a stranger needs assurance that a real organisation stands behind the request. A donor who may be an individual at home should not have their exact location broadcast to anyone who signs up.

The problem addressed by this project is therefore to *design and implement a web-based platform that lets a donor post surplus food with a hard pickup deadline, directs it to nearby verified organisations through an explainable ranking, coordinates a volunteer courier through pickup and delivery, and records every step as server-stamped evidence, while ensuring that each participant can see and do only what its role and its relationship to a donation permit.*

1.1.2 Motivation

Four observations motivate the design.

Coordination, not supply, is the bottleneck. Surplus food and recipient organisations already exist. A modest reduction in the delay between posting and acceptance directly converts food that would have been discarded into served meals.

This is why the proposal’s primary evaluation metric is *time-to-claim* [GSHS26], and why the system treats the pickup deadline as an absolute timestamp rather than a display string.

Trust must be earned, not asserted. Anyone can register as an organisation. If self-registration were enough to receive food, the platform would hand food, and donors’ addresses, to unvetted accounts. FoodLink therefore separates *holding a role* from *being trusted*. An administrator verifies each organisation, and that verification gates ranking, acceptance and access to the open donation pool.

Decisions should be explainable. A recipient deciding whether to accept, a donor wondering why an organisation was suggested, and an evaluator checking the system should all be able to see *why* a ranking came out as it did. A published weighted sum with per-criterion scores and plain-language reasons meets that requirement. An opaque learned model would not, and would in any case need outcome data the project does not yet have.

Evidence should not be self-reported. Metrics such as time-to-claim and rescue rate are only credible if their timestamps come from the server at the moment of each transition. FoodLink records every lifecycle change in an append-only ledger (`status_events`) and derives its metrics from it.

As a software engineering course project, the work also aims to demonstrate sound engineering practice: explicit requirements, a documented architecture and decision log, automated tests for every security and integrity property, and continuous integration.

1.1.3 Existing System

In the setting the proposal describes, redistribution happens through personal phone calls, messaging groups and ad-hoc arrangements between a kitchen and an organisation it already knows [GSHS26]. There is no shared software system. Table 1.1 summarises the limitations of this informal practice and how FoodLink responds to each. The comparison is with informal coordination as a practice; this report makes no claims about specific third-party products.

1.1.4 Proposed System

FoodLink is a three-tier web application. A React single-page application talks to a REST API written in Python with FastAPI, which persists data through SQLAlchemy to a relational database (SQLite by default). The API is the single authority for every rule. The browser renders what the API permits and never decides access or lifecycle legality itself.

The end-to-end flow, as implemented, is:

1. A **donor posts** a donation: food name, category, quantity and unit, storage

Aspect	Informal coordination	FoodLink (implemented)
Visibility of surplus	Known only to the contacts the donor calls	One open pool visible to every verified organisation
Choice of recipient	Whoever answers first	Ranked list of verified, nearby organisations, with reasons
Deadline	Mentioned verbally, not enforced	Absolute timestamp; overdue food leaves the pool and cannot be accepted
Trust	Personal familiarity	Administrator verification gates ranking, pool access and acceptance
Courier	Arranged separately, untracked	Claimable pickup task bound to one courier by a conditional database write
Status	Unknown after handover	Nine-state lifecycle visible to every party
Records	None	Append-only, server-stamped event ledger; derived metrics
Privacy	Addresses shared freely	Exact pickup pin withheld from a courier until the courier claims the run

Table 1.1: Informal coordination compared with the implemented FoodLink system

type, description, an optional photograph, the pickup address and map coordinates, an optional preparation time and an absolute pickup deadline. The server rejects a deadline that has already passed.

2. The server stores the donation as **AVAILABLE**, stamps an **AVAILABLE** event, and immediately **ranks** every recipient organisation. An organisation is eligible only if it is verified, has map coordinates and lies within an 8 km service radius. If any is eligible, the donation moves to **MATCHED**, and the leading score is frozen on the donation. A match is a suggestion: nothing is assigned.
3. Every **verified organisation** sees the donation in its open pool while the deadline has not passed, together with its own live match score and explanation. One organisation **accepts**, and the donation becomes **ACCEPTED**.
4. The donation appears in the unclaimed queue of every **volunteer courier**. There the pickup location is shown only as a coarse area of roughly one square kilometre. One courier **claims** it (**VOLUNTEER_ASSIGNED**) and only then sees the exact pin, address and donor name.

5. The courier **collects** the food (**PICKED_UP**) and **delivers** it (**DELIVERED**).
6. The accepting organisation **confirms receipt** (**COMPLETED**). The organisation's completion count and the courier's delivery count are incremented, which in turn feeds the reliability criterion of future rankings.
7. Along the way, the donor (before pickup) or an administrator may **cancel**, the accepting organisation may **release** a claimed pickup back to the courier queue, and an administrator may run the **expiry sweep** that closes overdue, unaccepted donations as **EXPIRED**. Every transition appends a server-stamped event.

Six principles run through the implementation and recur throughout this report: the server is the only authority; lifecycle rules are data rather than scattered conditionals; rankings are explainable; trust comes from administrator verification; each reader is told only what its relationship to a record justifies; and history is append-only.

1.1.5 Scope

In scope and implemented at mid-term. Account registration and sign-in for the donor, organisation and courier roles, and command-line and administrator-only creation of administrators. Role-, ownership-, lifecycle- and verification-based authorization. Donation posting with a browser-resized photograph. Explainable recipient ranking. The full nine-state donation lifecycle, with conditional (race-safe) status writes and an append-only event ledger. Courier claim, pickup and delivery. Receipt confirmation. Standing requirements posted by organisations, and a donor needs board. Organisation verification. The manual expiry sweep. Platform metrics. Four role-specific web portals. Rate limiting of sign-in, registration and donation posting. Automated backend and frontend test suites run in continuous integration.

Partially implemented. Cancellation, pickup release and administrator account management are implemented and tested in the API but are not yet offered in the web portal. The same applies to an organisation setting its map coordinates, which it needs in order to be ranked. Phone-layout screens exist under the `/m/*` routes but are reachable only by typing the address.

Out of scope, or not yet built. Email, SMS or push notifications; maps, road distances or route planning; object storage for photographs (they are stored inline in the database row); a scheduler for the expiry sweep; any deployment or hosting; a real-world pilot with users; machine learning or computer vision; and any certification of food safety. As the proposal states, the platform coordinates handovers and records the information a recipient needs; it does not vouch for the food [GSHS26].

1.2 Project Objectives

1. **Objective 1:** Allow a donor to post surplus food with an absolute pickup deadline, validated and stored by the server, within a single short form.
2. **Objective 2:** Rank eligible recipient organisations for each donation with a transparent, published weighted heuristic whose per-criterion scores and reasons are returned with every result.
3. **Objective 3:** Enforce a complete donation lifecycle, from posting to completion, cancellation or expiry, in which illegal transitions and concurrent conflicting transitions are refused, and every transition is recorded as a server-stamped event.
4. **Objective 4:** Restrict every read and write by role, ownership, lifecycle state and administrator-granted trust, and withhold sensitive data (such as a donor's exact pickup location) from readers who do not need it.
5. **Objective 5:** Make demand visible before supply by letting organisations post standing requirements that donors can browse.
6. **Objective 6:** Derive the proposal's evaluation metrics (median time-to-claim, rescue rate, expiry-loss rate and median handover time) from the event ledger rather than from self-reported data.
7. **Objective 7:** Protect the implementation with automated tests and continuous integration, so that every security and integrity property is exercised on each change.
8. **Objective 8:** Prepare the system for deployment and a pilot with real donors, organisations and couriers.

To make the objectives SMART (Specific, Measurable, Achievable, Relevant and Time-bound) for evaluators, Table 1.2 attaches a measurable criterion to each and records its verified status at the mid-term point.

1.3 Organisation of the Report

The report follows the chapter structure of the TIET report template [Tha26] and adds two chapters that this evaluation requires. Chapter 2 presents the methodology, architecture, requirements, database, API and frontend design, and then documents the implementation module by module. Chapter 3 describes how the system was tested and what the results show, including its current limitations. Chapter 4 is

No.	Measurable criterion	Status at mid-term
1	POST <code>/api/donations</code> stores a donation and refuses a past deadline	Met (Section 2.2.1)
2	Five weighted criteria, gates and reasons returned by <code>/matches</code> and <code>viewerMatch</code>	Met (Section 2.2.2)
3	Nine states; 409 on illegal or lost transitions; one event per transition	Met; covered by lifecycle and concurrency tests
4	Role, ownership, lifecycle and verification checks on every route; scoped reads	Met; covered by read-scope and authorization tests
5	Organisations create, revise, retire and reopen needs; donors read the verified board	Met (Section 2.2.4)
6	GET <code>/api/metrics</code> computes all four figures from <code>status_events</code>	Instrumented; not yet measured on real use
7	Backend and frontend suites pass in CI on every push	Met: 495 backend and 156 frontend tests pass (Chapter 3)
8	A staging deployment and a pilot by the final evaluation (week 17)	Not started (Chapter 4)

Table 1.2: Project objectives, measurable criteria and verified mid-term status

the separate Feasibility Report, covering technical, operational, economic, schedule, and legal, security and privacy feasibility. Chapter 5 summarises the work, the key findings and future work. Chapter 6 closes the report with the UML design artefacts: the use case diagram with detailed use case scenarios, the class diagram and the activity diagram.

Methodology and Design

FoodLink was built incrementally, in the iterative style the proposal committed to [GSHS26; Som16]. Work was organised as numbered tasks, the most recent being Task 46. Each task was small enough to review on its own and carried its own regression tests. Every non-trivial technical choice was recorded with its reasoning and constraints in a decision log. That log currently holds entries D-01 to D-61, and this chapter cites entries by number where they explain a design choice [Foo26a]. Three audits were run against the working system: a quality audit on 2 September 2026, and health audits on 5 and 10 September 2026. Their findings were prioritised from P0 (stop everything) to P3 (backlog) and fixed in that order. The five P1 findings of the latest audit are all fixed (Section 3.3). Continuous integration runs the complete backend and frontend test suites on every push.

2.1 System Architecture

2.1.1 High-Level Design

FoodLink follows a classic three-tier architecture, shown in Table 2.1. The *presentation tier* is a React single-page application running in the browser. The *application tier* is a stateless REST API implemented with FastAPI and served by uvicorn. The *data tier* is a relational database reached only through the SQLAlchemy object-relational mapper; no hand-written SQL appears in the application code.

Tier	Responsibility	Principal source files
Presentation (browser)	Role portals, forms, route guards, photo resizing, rendering what the API returns	<code>frontend/src/App.tsx</code> , <code>context/</code> , <code>pages/</code> , <code>mobile/</code> , <code>lib/api.ts</code>
Application (API)	Authentication, authorization scopes, validation, lifecycle rules, ranking, rate limiting, metrics	<code>code/foodlink/main.py</code> , <code>routers/</code> , <code>security.py</code> , <code>matching.py</code> , <code>serialize.py</code> , <code>ratelimit.py</code>
Data	Six tables, schema migrations, UTC time handling	<code>models.py</code> , <code>database.py</code> , <code>migrate.py</code> , <code>code/migrations/</code>

Table 2.1: Architectural tiers of FoodLink and where each is implemented

Backend layering. Inside the API, dependencies point one way: *routers* call *shared pure modules*, and both use the *ORM*. The five routers are `auth`, `donations`, `organisations`, `admin` and `metrics`. The shared modules are `matching`, `serialize`, `security` and `ratelimit`. The project deliberately has no separate service or repository layer (D-07). Logic that is genuinely reusable and independent of HTTP (the ranking arithmetic, the serializer that applies disclosure rules, token handling and rate limiting) was extracted into modules, while request orchestration stays in the router functions. The matcher in particular holds no database session, which is why it can be unit-tested directly.

Read path. When a signed-in user opens a page, the frontend's single HTTP client (`lib/api.ts`) attaches the stored bearer token and calls, for example, `GET /api/donations`. FastAPI resolves the dependency chain before the handler runs: (i) `get_db` opens a database session; (ii) `get_current_user` verifies the token's signature and expiry, then re-reads the user row, refusing a deleted or suspended account; (iii) where a route is role-restricted, `require_roles` checks the role held in that row. The handler then asks a scope helper (`_readable_by`) for a SQL `WHERE` clause describing exactly which donations this caller may read, and applies it inside the query. Related rows are loaded with `selectinload`, so a page of donations costs a fixed number of queries rather than one per row. Finally `serialize.donation_out` converts each row into the camelCase JSON shape, withholding any field the caller's relationship to the donation does not justify. The frontend's `lib/adapters.ts` translates that JSON into the domain types the components render.

Write path. A button such as *Accept* calls a mutation in `AppContext`, which posts to the API. The handler validates the body against a Pydantic schema. It then checks, in order, the lifecycle transition table, the caller's role, the caller's ownership of the donation and any target-specific rules. It applies side effects, performs a *conditional* status write, appends a `StatusEvent`, and commits everything in a single transaction. The client then reloads its data from the server rather than patching its local copy (D-11), so the browser can never disagree with the server about the lifecycle state.

External dependencies and background work. The backend makes no out-bound network requests. It has no email, SMS, push, map, payment or object-storage integration, and no scheduler, queue or worker process. In development, Vite's server proxies `/api` to the backend on port 8000, so the browser talks to a single origin. In any other arrangement, a CORS allowlist (by default `http://localhost:5173` and `http://127.0.0.1:5173`) decides which origins the browser may call from.

2.1.2 Technical Stack

- **Framework/Language:** Python 3 with FastAPI for the REST API [Ram26] (developed on Python 3.13; version 3.10 or later is required). TypeScript with React 18 for the web client [Met26], built with Vite [Vit26a].
- **Database:** SQLite by default, through SQLAlchemy 2.0 with typed mappings [SQL26b; SQL26c]. Schema changes are managed by Alembic [SQL26a]. The code is written to run on PostgreSQL through the `DATABASE_URL` setting, but that configuration is untested and no PostgreSQL driver is pinned.
- **Tools:** pytest with FastAPI's test client for the backend [pyt26]; Vitest with Testing Library and jsdom for the frontend [Vit26b]; GitHub Actions for continuous integration; Git for version control.
- **Libraries:** Pydantic 2 for validation and serialization [Pyd26]; PyJWT for signed tokens; bcrypt for password hashing; uvicorn as the ASGI server; React Router 6 for routing; Tailwind CSS 3 for styling; lucide-react for icons.

Table 2.2 lists the exact version constraints declared in `code/requirements.txt` and `frontend/package.json`.

2.1.3 System Requirements

Software requirements. To run FoodLink locally:

- Python 3.10 or later (3.13 is used in development and CI), with the packages in `code/requirements.txt`;
- Node.js with npm (CI uses Node.js 20), with the packages in `frontend/package.json`;
- no separate database server, since SQLite is used by default and the schema is migrated automatically when the API starts;
- a signing key in `FOODLINK_SECRET_KEY` (at least 32 characters). For local development only, `FOODLINK_DEV_INSECURE_SECRET=1` selects a known development key and makes the server print a warning. Without one of the two, the backend, the command-line tool and Alembic all refuse to start (D-22);
- a current web browser for the portals.

Hardware requirements. The repository defines no minimum hardware specification, and none has been measured. The system has so far been developed and tested on ordinary developer laptops and on GitHub-hosted Ubuntu CI runners. Constraints relevant to future hosting are discussed in the Feasibility Report (Section 4.1).

Component	Constraint	Role in FoodLink
FastAPI	≥ 0.115	Routing, dependency injection, OpenAPI docs at <code>/docs</code>
uvicorn[standard]	≥ 0.30	ASGI server
SQLAlchemy	≥ 2.0	ORM with typed <code>Mapped</code> columns
Alembic	≥ 1.13	Migrations; one revision, <code>ae4636b1e6d4</code>
Pydantic[email]	≥ 2.9	Request/response schemas, field bounds, email validation
PyJWT	≥ 2.9	HS256 access tokens
bcrypt	≥ 4.2	Password hashing
python-multipart	$\geq 0.0.9$	Form-encoded login body
React / React DOM	$\sim 18.3.1$	User interface
React Router DOM	$\sim 6.26.0$	Client-side routes and portal guards
TypeScript	$\sim 5.5.3$	Static typing (<code>tsc</code> in the build)
Vite	$\sim 5.4.2$	Development server, proxy and production bundle
Tailwind CSS	$\sim 3.4.10$	Utility-first styling
Vitest / Testing Library	$\sim 3.2.7$ / $\sim 16.3.3$	Frontend unit and component tests

Table 2.2: Technology stack with the versions declared in the repository

2.1.4 Functional Requirements

Tables 2.3 and 2.4 list the functional requirements that the implementation satisfies, derived from the proposal and verified against the routes and portals. “API” in the status column means the behaviour is implemented and tested in the API but has no control in the web portal yet.

2.1.5 Non-Functional Requirements

Table 2.5 lists the quality attributes the design targets, the concrete mechanism that addresses each, and any known gap.

2.1.6 Database Design

The schema has six tables, created by a single Alembic revision (`ae4636b1e6d4`) and defined in `code/foodlink/models.py`. Table 2.6 summarises them. The class

ID	Requirement	Status
FR-01	A visitor can register as a donor, a recipient organisation or a courier; registering as an administrator is refused. An organisation or courier profile is created with the account.	Implemented
FR-02	A user signs in with email and password and receives a bearer token; a suspended account is refused with an explanatory message.	Implemented
FR-03	A user can edit their name, organisation and phone number and change their password.	Implemented
FR-04	A donor posts a donation with food name, category, quantity, unit, storage type, description, optional photo, pickup address and coordinates, optional preparation time and a future pickup deadline.	Implemented
FR-05	On posting, the system ranks eligible organisations and records MATCHED if at least one is eligible, freezing the leading score.	Implemented
FR-06	A verified organisation browses the open pool (not yet accepted and not past its deadline), with its own live match score and reasons.	Implemented
FR-07	A reader of a donation can retrieve the ranked list of eligible organisations with per-criterion scores and reasons.	Implemented (API; mobile)
FR-08	A verified organisation accepts an open donation on its own behalf; an overdue donation cannot be accepted.	Implemented
FR-09	A courier browses unclaimed accepted pickups (coarse location only) and claims one; only one courier can win a claim.	Implemented
FR-10	The assigned courier records collection and delivery.	Implemented
FR-11	The accepting organisation confirms receipt, completing the donation and updating completion counters.	Implemented

Table 2.3: Functional requirements FR-01 to FR-11 and implementation status

ID	Requirement	Status
FR-12	The donor cancels their donation before collection; an administrator can also cancel after collection.	Implemented (API)
FR-13	The accepting organisation releases a claimed pickup back to the courier queue.	Implemented (API)
FR-14	An organisation posts, revises, retires and reopens standing requirements; donors browse active needs of verified organisations.	Implemented
FR-15	An organisation maintains its profile (name, type, address, map pin, capacity, contact); changing its name or pin voids its verification.	Implemented (pin: API only)
FR-16	A courier sets availability and base location.	Implemented
FR-17	An administrator verifies or revokes organisations.	Implemented
FR-18	An administrator runs the expiry sweep, closing overdue unaccepted donations.	Implemented
FR-19	An administrator creates accounts of any role and suspends, restores, renames or re-roles accounts, without locking out the last administrator.	Implemented (API)
FR-20	Every lifecycle transition is recorded with previous state, new state, actor, note and server time, and is shown as a timeline.	Implemented
FR-21	Platform metrics (volumes, median time-to-claim, median handover time, rescue rate, expiry-loss rate) are computed from the event ledger.	Implemented
FR-22	An administrator exports the filtered donation list as CSV.	Implemented

Table 2.4: Functional requirements FR-12 to FR-22 and implementation status

diagram in Figure 6.2 shows every attribute and multiplicity.

Several design choices shape this schema.

- **Single user table, satellite profiles.** Roles are an enumerated column, not a class hierarchy. Registering as an organisation creates a **recipients** row, and registering as a courier creates a **volunteers** row, so each account is ready to act immediately.
- **Events instead of timestamp columns (D-01).** A column such as `accepted_at` could record neither who acted nor a state entered twice (a released pickup re-enters `ACCEPTED`). The ledger records both, and adding a state needs no migration. `occurred_at` is never accepted from a client, which is what makes the metrics attributable.

Attribute	How it is addressed	Known gap
Security	bcrypt password hashes; signed, expiring JWTs; user row re-read on every request; role, ownership, lifecycle and trust checks on the server; fail-closed signing key	No token revocation, refresh or MFA
Privacy	Read scopes applied in SQL; exact pickup pin, address and donor name withheld from unclaimed couriers; organisation distances blurred for other readers	Description, photo and notes are bounded but not scoped
Data integrity	Transition table; every status write conditional on the expected current state; single commit per request; UTC-normalised timestamps	Expiry sweep writes without the guard; SQLite foreign keys not enforced
Explainability	Published weights; per-criterion scores and plain-language reasons with every match	Weights are judgement, not tuned on data
Auditability	Append-only, server-stamped <code>status_events</code> ledger with actor	No admin audit log for account changes
Abuse resistance	Sliding-window rate limits on login, registration and donation posting; bounded text and image sizes	Limits are per process
Usability	Short forms; browser-side photo resizing; human-readable error sentences shown verbatim; role-specific portals	No notifications; phone layout reachable only by URL
Maintainability	Typed models and schemas; rules expressed as data; decision log; 651 automated tests	Frontend lint script inactive (no ESLint)
Portability	SQLite by default; all data access through SQLAlchemy	PostgreSQL untested
Performance	Eager loading avoids per-row queries; list size capped at 500; input and image caps	Every write re-downloads all lists; recipients filtered in Python

Table 2.5: Non-functional requirements, mechanisms and known gaps

Table	Purpose and key columns	Relationships
users	One table for every account: name , unique email , password_hash , role (enumeration <code>UserRole</code>), optional organization and phone , is_active , created_at	Has at most one recipient profile and at most one courier profile; posts donations; acts in status events
recipients	A kitchen, shelter or NGO: name , type , location , optional latitude/longitude , capacity (meals), contact person and phone, is_verified , counters accepted_donations and completed_donations	Optionally owned by a user; receives donations; owns requirements (cascade delete)
volunteers	A courier profile: base location , optional coordinates, is_available , completed_deliveries , rating	Owned by exactly one user (unique foreign key); carries donations
donations	The core record: food details, optional inline image_url , pickup location with exact coordinates, prepared_at , absolute pickup_deadline , status (enumeration <code>DonationStatus</code>), frozen match_score , created_at	Belongs to its donor; recipient_id is null until accepted; volunteer_id is null until claimed; owns its events (cascade delete)
status_events	Append-only history: from_status , to_status , actor_id , note , and occurred_at stamped by the server	Belongs to one donation; actor is a user, or null for the expiry sweep
requirements	A standing need: food_type , quantity_needed , unit , beneficiary_count , urgency , daily_recurring , notes , is_active , created_at	Belongs to one recipient organisation

Table 2.6: Database tables, purpose and relationships

- **Derived values stay derived (D-15).** An organisation's reliability score is a computed property. It is 85 until the organisation has three accepted donations, then $100 \times \text{completed/accepted}$. Distance is computed from two coordinate pairs, never stored. The one deliberate exception is `match_score`, frozen so that the number shown later is the one a decision was actually made on (D-30).
- **Counters maintained by the lifecycle.** The acceptance and completion counters are updated inside the same transaction as the transition that justifies them. A cancellation takes a counted acceptance back out (D-58), so a donor's or administrator's cancellation never lowers an organisation's reliability.
- **Time zones.** A custom column type, `UtcDateTime`, normalises every timestamp to timezone-aware UTC on write and re-attaches UTC on read. SQLite has no timezone type, and a zoneless deadline would otherwise be read by browsers as local time (D-09).
- **Invariants in application code.** The state machine, coordinate ranges and counters are enforced by the application, not by database constraints, and SQLite does not enforce foreign keys by default. The lifecycle's own writes nevertheless carry their preconditions in SQL (Section 2.2.1).

2.1.7 API and Backend Design

The API is a JSON REST interface under the `/api` prefix. Python code uses snake_case, while wire fields are camelCase through a Pydantic alias generator; for example, `food_name` travels as `foodName`. Interactive OpenAPI documentation is served at `/docs`. Tables 2.7 and 2.8 list all 27 routes.

Conventions. The API answers consistently, and the frontend relies on that consistency.

- **Errors are sentences.** Every refusal carries a human-readable `detail`, which the frontend shows verbatim (D-18). Sign-in uses one message for both an unknown email and a wrong password, so the response does not reveal which addresses have accounts.
- **Status codes carry meaning.** 401 means the token is invalid or the account is unusable. 403 means the role or trust level is insufficient. 404 means not found *or not visible to you*. 409 means the donation is not in a state that allows the request. 422 means the input is invalid. 429 means a rate limit was hit, and carries a `Retry-After` header.

Method	Path	Who	Purpose
POST	/api/auth/register	Anonymous	Create donor, organisation or courier account (10/hour per IP)
POST	/api/auth/login	Anonymous	Form-encoded email and password → token (30 per 5 min per IP)
GET, PATCH	/api/auth/me	Any signed-in	Read or edit own account
POST	/api/auth/password	Any signed-in	Change password (current one required)
POST	/api/donations	Donor, admin	Post and rank a donation (donors: 10/hour per account, 30/hour per IP)
GET	/api/donations	Any signed-in	Scoped list; status , mine , limit ≤ 500
GET	/api/donations/{id}	Any signed-in	One donation, same scope (404 outside it)
GET	/api/donations/{id}/matches	Readers of the donation	Ranked organisations, limit ≤ 25
POST	/api/donations/{id}/status	Per target state	Drive a lifecycle transition (optional note)
GET	/api/recipients	Any signed-in	Scoped organisation directory
GET, PATCH	/api/recipients/me	Organisation	Read or edit own organisation
GET	/api/requirements	Any signed-in	Scoped needs; includeInactive
POST	/api/requirements	Organisation, admin	Post a standing need for own organisation
PATCH	/api/requirements/{id}	Owning organisation	Revise, retire or reopen
GET	/api/volunteers	Admin, organisation	Scoped courier roster
GET, PATCH	/api/volunteers/me	Courier	Read or edit own courier profile

Table 2.7: REST API routes, part 1: authentication, donations and organisations

Method	Path	Who	Purpose
GET	/api/metrics	Any signed-in	Ledger-derived platform metrics
GET, POST	/api/admin/users	Admin	List accounts; create an account of any role
PATCH	/api/admin/users/{id}	Admin	Suspend, restore, rename, re-role
POST, DELETE	/api/admin/recipients/{id}/verify	Admin	Verify or revoke an organisation
POST	/api/admin/maintenance/expire	Admin	Expiry sweep
GET	/api/health	Anonymous	Liveness (no database access)

Table 2.8: REST API routes, part 2: metrics, administration and health (27 routes in total; paired methods share a row)

- **Denial never confirms existence.** A donation outside the caller’s scope returns the same 404 as one that does not exist (D-24), and a directory the caller may not read returns an empty list (D-26).
- **PATCH has three states per field (D-61).** An omitted field is untouched. An explicit `null` clears a nullable column, such as a pin or phone number, and leaves a mandatory column unchanged rather than causing a server error.
- **Privileged fields are absent from self-service schemas (D-16).** For example, `RecipientUpdate` has no `is_verified` field, and `ProfileUpdate` has no `role`.

Backend modules. `main.py` creates the application, installs CORS and the routers, and runs `ensure_schema_current()` at start-up, which applies any outstanding Alembic revision. A database that predates migrations is reported rather than rewritten (D-23). `config.py` reads settings from the environment; the most important is the fail-closed signing key. `cli.py` provides `create-admin`, `promote`, `reset-password` and `list-admins`. The first administrator can only be created this way, because the API has no route to one (D-04). `seed.py` loads demonstration data.

2.1.8 Frontend Architecture

The web client is a React single-page application with a small, deliberate set of seams.

- **One HTTP boundary (D-12).** `lib/api.ts` is the only module that calls `fetch`. It attaches the token, turns error responses into an `ApiError` carrying the server's sentence, and signals a 401 to the whole application at once. `lib/adapters.ts` maps wire types to the domain types the components use.
- **Two context providers.** `AuthContext` keeps the token in `localStorage` and re-fetches the account from `/api/auth/me` on start-up, so a suspended or re-rolled account cannot act on a stale copy of its own permissions (D-13). `AppContext` loads the donation list (up to 500 rows) and the role-appropriate slices (requirements, organisations, couriers, metrics) once per sign-in. It reloads them after every write (D-11). A `useAction` hook tracks in-flight state and shows success or error toasts.
- **Routing and guards.** React Router mounts four portals: `/donor`, `/ngo`, `/volunteer` and `/admin`. Each sits behind a `ProtectedRoute` that redirects users away from portals their role does not use; administrators may enter every portal. These guards are a user-experience convenience, not security. The API independently refuses anything the role may not do (D-14).
- **Utilities.** `lib/geo.ts` chooses which distance or location text to display, including the coarse pickup area shown to couriers. `lib/time.ts` computes urgency from the absolute deadline. `lib/impact.ts` computes each account's own impact figures from its own donations (D-32). `lib/image.ts` decodes a chosen photo, scales its long edge to at most 1280 px and re-encodes it as JPEG at decreasing quality until it fits the server's 256 KiB cap (D-56).
- **Styling.** Tailwind CSS with a customised warm palette and typefaces (D-10), and lucide-react icons.
- **Phone layouts.** A parallel set of phone-layout screens under `src/mobile/` is mounted at `/m/*`. Nothing links to them yet, so they are reached by typing the address (D-20).

2.2 Implementation Details

This section documents the implementation module by module. Modules 1 to 3 form the core functionality: the lifecycle, the matching and the access model. Modules 4 to 6 are the secondary features built on them.

2.2.1 Module 1: Core Functionality — Donation Lifecycle

The donation lifecycle is the core of FoodLink. It lives in `routers/donations.py`, with the state graph in `models.py`. It has nine states (Table 2.9) and is governed by

two tables that are plain data rather than scattered conditionals (D-02). **ALLOWED_TRANSITIONS** says which moves are legal at all; any other move is answered with 409. **TRANSITION_ROLES** says which roles may cause each target state; any other role is answered with 403.

State	Meaning
AVAILABLE	Posted; no eligible organisation was ranked at posting. Open to verified organisations until the deadline.
MATCHED	Posted and at least one eligible organisation was ranked. Still open: a match is a suggestion, not a claim.
ACCEPTED	Bound to one organisation; waiting for a courier to claim the pickup.
VOLUNTEER_ASSIGNED	Bound to one courier, who can now see the exact pickup location.
PICKED_UP	The courier has collected the food.
DELIVERED	The courier has handed the food over.
COMPLETED	The organisation has confirmed receipt. Terminal.
CANCELLED	Withdrawn by the donor or an administrator. Terminal.
EXPIRED	Closed after the deadline passed without acceptance. Terminal.

Table 2.9: Donation lifecycle states

ALLOWED_TRANSITIONS also permits **MATCHED** → **AVAILABLE**, but no role is listed for it, so it cannot currently be performed.

Donation creation

POST /api/donations is open to donors and administrators. Before the handler runs, a dependency applies two sliding-window rate limits to donors: 10 posts per hour per account and 30 per hour per client address, each with its own 429 sentence. Administrators are exempt (D-59). The Pydantic schema bounds every field to its column size. Free text such as the description is capped at 2,000 characters (D-60). The quantity must be positive, the coordinates must be valid, and **imageUrl** must be either a base64 image data URL of at most 262,144 characters or an **http(s)** link (D-56). The handler then refuses a deadline that is not in the future, stores the donation as **AVAILABLE** and appends the first event. It loads every recipient organisation with its active requirements, ranks them (Section 2.2.2), and if any is eligible freezes the leading score and records **MATCHED** with the note “Top match: *name*”. Everything commits once, and the response is serialized under the caller’s scopes.

Transition	Allowed roles	Conditions and side effects
(new) → AVAILABLE	Donor, admin	Deadline in the future; donor rate limits; valid, bounded input. Appends the first event.
AVAILABLE → MATCHED	Automatic at posting; admin	Recorded when ranking finds an eligible organisation; freezes the leading score; the event note names the top match.
AVAILABLE/ MATCHED → ACCEPTED	Organisation, admin	Deadline not passed (else 409). An organisation accepts only for itself; an administrator names one. The organisation must be verified (else 403). Binds it, adds one counted acceptance, freezes its score.
ACCEPTED → VOLUNTEER_ASSIGNED	Courier	Caller needs a courier profile (else 422); a conditional claim binds the courier (loser gets 409). Administrators are listed but have no courier profile, so always receive 422.
VOLUNTEER_ASSIGNED → ACCEPTED (release)	Accepting organisation, admin	Owned; the organisation must still be verified; clears the courier; not counted as a new acceptance and does not re-freeze the score (D-41).
VOLUNTEER_ASSIGNED → PICKED_UP	Assigned courier, admin	Owned by the courier bound to the run.
PICKED_UP → DELIVERED	Assigned courier, admin	Owned by the courier bound to the run.
DELIVERED → COMPLETED	Accepting organisation, admin	Owned; increments the organisation's completed donations and the courier's completed deliveries.
Any of the first four states → CANCELLED	Owning donor, admin	Owned; if an organisation was bound, its counted acceptance is withdrawn in the same transaction (D-58).
PICKED_UP → CANCELLED	Admin only	A donor receives 409 once the food is collected; the acceptance is withdrawn.
AVAILABLE/ MATCHED/ACCEPTED → EXPIRED	Admin	Through the status route, or in bulk through the expiry sweep, which covers only overdue AVAILABLE and MATCHED donations.

Table 2.10: Lifecycle transitions, permitted roles and enforced conditions

The transition algorithm

Every other transition goes through one endpoint, `POST /api/donations/{id}/status`, whose handler `update_status` applies the checks in a fixed order:

1. load the donation (404 if the id does not exist);
2. refuse a move absent from `ALLOWED_TRANSITIONS` (409);
3. refuse a caller whose role is not in `TRANSITION_ROLES` for the target (403);
4. for *owned* transitions (`PICKED_UP`, `DELIVERED`, `COMPLETED`, `CANCELLED`, and `ACCEPTED` when it is a release), re-load the donation through the caller's own read scope. A caller who is not this donation's donor, courier or accepting organisation therefore receives the same 404 a read would give (D-34, D-35);
5. apply the target-specific rules and side effects described in Table 2.10;
6. perform the conditional status write and append the event, or refuse with 409;
7. for a cancellation, withdraw the counted acceptance;
8. commit once and return the donation as the caller is allowed to see it.

Concurrency: conditional writes

A lifecycle system that reads a row, checks it in Python and then writes it is vulnerable to *lost updates*. Two organisations can both read `AVAILABLE`, both conclude they may accept, and both succeed, which really happened before this was fixed [Kle17]. FoodLink closes this by making the check and the write the same database statement. The helper `_record` advances the status with

```
UPDATE donations SET status = :to WHERE id = :id AND status = :from
```

and treats a row count other than one as a conflict. It returns 409 with the state the row is now in, and appends no event. Because the write runs last before the single commit, a refused transition also rolls back its side effects, such as a counter increment or a frozen score (D-55). The courier claim uses a second conditional statement. It sets `volunteer_id` only if the status is still the expected one and the pickup is unclaimed (or already this courier's), so exactly one of several simultaneous claims wins (D-28). The alternative, `SELECT ... FOR UPDATE`, was rejected because SQLite silently ignores it. The fix would then have been untestable on the engine the project uses.

Acceptance, claim and release

An organisation always accepts as itself. If it names another organisation's id, it receives 403. Only an administrator may accept on an organisation's behalf. Verification is re-checked at acceptance, so an unverified organisation cannot take custody even if it obtains a donation's id (D-06). An overdue donation in the open pool cannot be accepted (409). This keeps behaviour identical whether or not the expiry sweep has run (D-50). Acceptance from `VOLUNTEER_ASSIGNED` is a *release*, not an acceptance: the courier is removed so that other couriers can claim the pickup, and neither the acceptance counter nor the frozen score changes (D-41). In the current web portal there is no release control; it is available through the API.

Cancellation and expiry

A donor may cancel their own donation in `AVAILABLE`, `MATCHED`, `ACCEPTED` or `VOLUNTEER_ASSIGNED`. Once the food is in a courier's hands (`PICKED_UP`), only an administrator may cancel. If an organisation was bound, the cancellation decrements its acceptance counter with a single conditional SQL statement floored at zero. As a result, a cancellation it did not cause cannot lower its reliability score (D-58). The expiry sweep (`POST /api/admin/maintenance/expire`) closes every `AVAILABLE` or `MATCHED` donation whose deadline has passed. It appends an event with no actor and the note "Deadline passed with no recipient", and returns the number expired. Nothing schedules it; an administrator triggers it from the donations page.

The event ledger and metrics

Each `StatusEvent` records the previous and new state, the acting user, an optional note (bounded to 2,000 characters) and a server timestamp. The donation detail screens render these rows as a timeline. `GET /api/metrics` computes the proposal's evaluation figures from the same rows (Section 2.2.5).

2.2.2 Module 2: Core Functionality — Recipient Matching and Explainability

`matching.py` ranks recipient organisations for a donation. It is a transparent weighted-sum heuristic and deliberately *not* a learned model: the proposal committed to an explainable ranking that a recipient can read and a marker can verify by hand (D-05). The module is pure Python with no database access; the routers load the rows and pass them in.

Eligibility gates

Before any score is computed, `score_pair` excludes an organisation entirely (returns nothing) if any of three gates fails (D-06): (i) it is not verified by an administrator; (ii) it has no map coordinates; (iii) its great-circle distance from the pickup pin exceeds the service radius R , which is 8 km by default (`MAX_MATCH_RADIUS_KM`). Gating rather than scoring low is intentional. A suggestion the organisation is not allowed to act on would be a false promise to the donor.

Scoring criteria

Let d be the haversine distance in kilometres between the pickup pin and the organisation [Sin84], q the donation’s quantity, c the organisation’s capacity in meals, a and k its accepted and completed donation counts, and m the minutes remaining until the pickup deadline. Each criterion yields an integer from 0 to 100 (“round” denotes Python’s `round`, which sends exact halves to the nearest even integer).

Distance decays linearly to zero at the radius:

$$S_D = \begin{cases} \text{round}(100(1 - d/R)) & d < R \\ 0 & d \geq R \end{cases} \quad (2.1)$$

Quantity fit is assessed only when the donation’s unit is “Meals”, because capacity is counted in meals and the repository holds no conversion rule (D-42). With $r = q/c$, overfilling is penalised twice as steeply as underfilling:

$$S_Q = \begin{cases} \text{round}(40 + 60r) & r \leq 1 \\ \max(0, \text{round}(100 - 120(r - 1))) & r > 1 \end{cases} \quad (2.2)$$

Capacity headroom measures how much room the organisation keeps afterwards, in absolute meals, saturating at 100 spare meals:

$$S_C = \begin{cases} \text{round}(100 \min(1, (c - q)/100)) & c - q > 0 \\ 0 & c - q \leq 0 \end{cases} \quad (2.3)$$

For any other unit (Kg, Boxes, Pieces), $S_Q = S_C = 50$, which is identical for every candidate and so cannot reorder them, and the explanation says the size was not assessed.

Deadline comfort subtracts a rough travel estimate $t = 60d/20$ minutes (a flat 20 km/h, never shown to users as an arrival time) and treats two hours of slack as fully comfortable:

$$S_T = \begin{cases} \text{round}(100 \min(1, (m - t)/120)) & m - t > 0 \\ 0 & m - t \leq 0 \end{cases} \quad (2.4)$$

Reliability starts optimistic so that a new organisation is not permanently out-ranked by one with a single lucky completion:

$$S_R = \begin{cases} 85 & a < 3 \\ \text{round}(100k/a) & a \geq 3 \end{cases} \quad (2.5)$$

The overall score is the weighted sum, with the weights published in the **WEIGHTS** constant (Table 2.11):

$$S = \text{round}(0.25 S_D + 0.25 S_Q + 0.20 S_C + 0.15 S_T + 0.15 S_R) \quad (2.6)$$

Criterion	Weight	Question it answers
Distance	0.25	How close is the organisation to the pickup?
Quantity fit	0.25	Does the donation's size suit the organisation's capacity?
Capacity headroom	0.20	How much room will the organisation still have afterwards?
Deadline comfort	0.15	Can a collection fit comfortably before the food expires?
Reliability	0.15	How often has the organisation seen accepted donations through?

Table 2.11: Published weights of the five matching criteria (sum 1.00)

Ordering and the role of requirements

Candidates are sorted by a total order: overall score first, then a *requirement tier*, then requirement fit, then the urgency of that requirement, and finally the organisation id. An organisation's active standing requirements therefore break ties but never change a score (D-52). The best requirement for a donation is the one whose unit matches the donation's unit (no conversion) with the highest fit, where fit is 100 when the donation covers the stated quantity n and $\text{round}(40 + 60q/n)$ otherwise. A kitchen whose need can be measured ranks above one whose need cannot, which ranks above one with no posted need. Requirements are kept out of the score because the frozen `match_score` is visible to readers who may not read requirements. A score derived from them would leak private demand.

Explanations

Every result carries the five sub-scores and a list of reasons generated from fixed thresholds (Table 2.12). Distances are printed only when the reader is entitled to the true figure.

Condition	Reason text (as returned by the API)
$S_D \geq 70$	“ d km away in a straight line — well inside the collection radius” (or, with a blurred position, “Well inside the collection radius”)
Unit is not Meals	“Donation size not assessed: q unit cannot be compared with a capacity stated in meals”
$S_Q \geq 80$ (else $q > c$)	“ q meals fits the c -meal daily capacity closely” (else “Donation exceeds stated capacity by ...”)
$S_C \geq 70$	“ $c - q$ meals of capacity still free afterwards”
Requirement assessed	“Matches a standing need this kitchen posted for ...” or “Covers part of a standing need ...”
$S_T \geq 70$ / $S_T = 0$	“Comfortable margin before the pickup deadline” / “Deadline is too close for the estimated collection time”
$S_R \geq 85$	“ $S_R\%$ completion record on accepted donations”

Table 2.12: Threshold rules that generate the plain-language match explanation

Worked example

Table 2.13 shows the matcher’s actual output for a sample donation, obtained by calling `score_pair` from the repository on in-memory objects. The donation is 60 meals, room temperature, with a deadline three hours away. Kitchen A is new, 2 km away, with capacity 100. Kitchen B is 5 km away, with capacity 250 and 9 completions out of 10 acceptances. Kitchen C is 1 km away but unverified. Kitchen E is verified but 9 km away.

Organisation	d (km)	S_D	S_Q	S_C	S_T	S_R	S
Kitchen A	2.0	75	76	40	100	85	74
Kitchen B	5.0	38	54	100	100	90	72
Kitchen C (unverified)	excluded by the verification gate						
Kitchen E (9 km)	excluded by the radius gate						

Table 2.13: Matcher output for a 60-meal donation (computed with `score_pair`)

For Kitchen A, Equation 2.6 gives $0.25(75) + 0.25(76) + 0.20(40) + 0.15(100) + 0.15(85) = 73.5$, which rounds to 74. Its reasons were “2.0 km away in a straight line — well inside the collection radius”, “Comfortable margin before the pickup deadline” and “85% completion record on accepted donations”. Kitchen B ranks second despite a perfect headroom score, because distance and quantity fit carry more weight. The same donation posted in kilograms scores Kitchen A at 69: both size criteria fall to 50, and the explanation states that the size was not assessed.

Two scores and location privacy

A donation carries two different figures that must not be confused (D-30). `matchScore` is *frozen*. It is written at posting (the top candidate's score) and replaced at the first acceptance by the accepting organisation's own score. `viewerMatch` is *live*. It is the reading organisation's own ranking, computed on each request while the donation is still open, and it is what the organisation's *Available Donations* screen shows with its breakdown. Location-derived figures are restricted because scores and distances read back from pins of a donor's choosing could otherwise be used to triangulate an organisation's private address (D-45, D-47). When anyone other than the organisation itself (or an administrator) reads a ranking, the organisation's coordinates are first snapped to a 0.01° grid (roughly one kilometre), and `distanceKm` is returned as null. The frozen `matchScore` and `distanceKm` on a donation are visible only to administrators and to the organisation the figure describes. Eligibility itself is always decided on the true position, so the set of ranked organisations never depends on who is asking.

Design for future replacement

The router and the response shape depend only on `score_pair` and `rank_recipients`. If outcome data later justified a learned ranker, it could replace `score_pair` without changing the rest of the system. That is a design property, not an implemented capability.

2.2.3 Module 3: Core Functionality — Authentication, Authorization and Trust

Accounts and authentication

Registration (POST `/api/auth/register`) accepts a name, an email, a password of 8 to 128 characters and a role. The schema itself refuses the `admin` role, so the restriction is visible in the published API documentation (D-04). Registering as an organisation creates an *unverified* recipient row, optionally with type, address, coordinates and capacity. Registering as a courier creates a courier row. Passwords are stored as bcrypt hashes [PM99]. Sign-in uses the OAuth 2.0 password form, with the email in the `username` field [Har12]. It returns a JSON Web Token signed with HMAC-SHA256 [JBS15], carrying the user id, role and an expiry 720 minutes ahead. The browser keeps the token in `localStorage` and sends it as a bearer header. On every request, `get_current_user` verifies the token and *re-reads the user row*, ignoring the role claim. A suspension or role change therefore takes effect on the very next request (D-03). There is no refresh token, revocation list or multi-factor authentication. The signing key must be configured and at least 32 characters long; the service will not

start with a missing or known-public key (D-22).

Administrators

The first administrator can only be created from the command line (`python -m foodlink.cli create-admin`), because the API offers no anonymous path to that role. Later administrators are created by an existing administrator through `POST /api/admin/users`. Two lockout guards refuse an administrator suspending or demoting their own account, and refuse suspending or demoting the last active administrator.

Four layers of authorization

Every operation passes through up to four independent checks:

1. **Role:** `require_roles` or `TRANSITION_ROLES` (403).
2. **Ownership:** expressed as a SQL `WHERE` clause returned by a scope helper, so the database never returns rows the caller may not see (404 or an empty list).
3. **Lifecycle legality:** `ALLOWED_TRANSITIONS`, the donor cancellation window and the deadline rule (409).
4. **Trust:** `Recipient.is_verified`, set only by an administrator, and read by ranking, acceptance and the organisation's access to the open pool (D-37, D-53).

Table 2.14 lists the read scopes as implemented. Besides deciding *which* records a caller receives, FoodLink decides *what* a record says to that caller. An unclaimed pickup in a courier's queue carries a coarse `pickupArea` (the donor's pin snapped to the 0.01° grid) instead of the exact coordinates, address text and donor name. Claiming the run is what discloses them (D-57).

Verification as a trust boundary

Organisations begin unverified, except when an administrator creates the account, which counts as vouching for it. An unverified organisation is not ranked, cannot see the open pool and cannot accept. Its own donations remain readable to it. Verification is read live on every request, so revoking it closes the pool at once. Because verification vouches for *who* and *where* an organisation is, a real change to its name, latitude or longitude through `PATCH /api/recipients/me` clears the flag in the same commit, and an administrator must verify it again (D-54). Resubmitting unchanged values, or editing type, address text, capacity or contact details, does not affect it.

Data	Admin	Donor	Organisation	Courier
Donations (list, by id, matches)	All	Own	Own organisation's, plus the open pool only if verified	Unclaimed ACCEPTED plus own runs
Exact pickup pin, address, donor name	All	Own	All it can read	Own claimed runs only
Frozen score, true distance	All	None	Own organisation	None
Organisation directory (with phone)	All	None	Own row	None
Courier roster (with phone)	All	403	Couriers on own donations	403
Requirements	All	Active needs of verified organisations	Own (retired too, on request)	None
Requirement inputs to a ranking	All	Verified organisations	Own organisation	None

Table 2.14: Read and disclosure scopes by role, as implemented

Abuse limits and input bounds

Anonymous routes are limited per client address (30 sign-in attempts per 5 minutes; 10 registrations per hour). Donation posting is limited per donor account and per address (Section 2.2.1). All limits are sliding windows held in the server process's memory (D-27, D-59). Request schemas bound every donation and requirement text field to its column size, bound free text to 2,000 characters, and bound photos to 256 KiB of data URL. Account and organisation profile text fields are not yet bounded.

2.2.4 Module 4: Secondary Features — Standing Requirements

Standing requirements make demand visible before surplus appears. An organisation posts a need with a food type, quantity and unit, beneficiary count, an urgency label (the ranking recognises High, Medium and Low), a daily-recurring flag and notes (POST /api/requirements). The whole lifecycle is one flag, `is_active`, changed through PATCH /api/requirements/{id} (D-29). Setting it false retires a need,

whether it was met or no longer applies. The row is kept, and setting the flag true reopens it. Ownership is part of the query, so another organisation's requirement id returns 404. Reads are scoped (D-44, D-46): donors see the active needs of verified organisations on the *Needs Board*; an organisation sees its own, including retired ones when it asks with `includeInactive`; couriers see none. In ranking, an active requirement only breaks ties and adds a reason (Section 2.2.2). Food type, beneficiary count, recurrence and notes are not ranking inputs.

2.2.5 Module 5: Secondary Features — Administration and Platform Metrics

The administration router is protected by a single role dependency declared once for all of its routes. Through the web portal, an administrator can verify or revoke organisations (*Partner Organizations Directory*), browse and filter every donation, export the filtered list as CSV, and trigger the expiry sweep (*Platform Donation Ledger*). The administrator can also view the courier roster and the analytics dashboard. Account management (listing, creating, suspending, restoring and re-rolling users) is implemented and tested in the API but has no screen yet. Because every entry in `TRANSITION_ROLES` includes the administrator, an administrator can also perform lifecycle transitions on a party's behalf, subject to the same legality rules.

`GET /api/metrics` loads every donation with its events and computes the following figures in Python:

- totals of donations, completed, active (non-terminal) and expired donations, organisations and couriers; and the sum of the quantities of completed donations (summed across units);
- **median time-to-claim**: minutes from a donation's creation to its first `ACCEPTED` event;
- **median handover time**: minutes from the first `ACCEPTED` event to the `DELIVERED` event;
- **rescue rate**: completed donations whose `COMPLETED` event is no later than the pickup deadline, as a percentage of donations that completed or expired;
- **expiry-loss rate**: expired donations as a percentage of those that completed or expired.

Any signed-in role may call the endpoint; only the administrator screens display it. The donor, organisation and courier impact pages compute figures from the account's own donations instead (D-32).

2.2.6 Module 6: Secondary Features — Role Portals and User Workflows

Table 2.15 maps each portal’s screens to the workflows they support. Every action button calls the API described above and then refreshes the data.

Portal	Routes	Workflow supported
Public	/, /login	Product landing page; sign-in and registration with role selection
Donor	/donor, /donor/create, /donor/donations, /donor/donations/: id, /donor/needs, /donor/impact, /donor/profile	Dashboard; post a donation with photo; list own donations and follow each one’s status timeline; browse the needs board; own impact; profile and password
Organisation	/ngo, /ngo/available, /ngo/accepted, /ngo/requirements, /ngo/impact, /ngo/profile	Dashboard; browse the open pool with the match breakdown and accept (shown only once verified); follow accepted donations and confirm receipt; manage standing needs; reliability and verification status; organisation profile (no map-pin field)
Courier	/volunteer, /volunteer/tasks, /volunteer/history, /volunteer/impact, /volunteer/profile	Dashboard; claim pickups from the unclaimed queue (coarse area), then record pickup and delivery; delivery history; availability and base location
Admin	/admin, /admin/donations, /admin/orgs, /admin/volunteers, /admin/analytics	Platform dashboard; all donations with filters, CSV export and expiry sweep; verify or revoke organisations; courier roster; ledger-derived analytics

Table 2.15: Web portals, their routes and the workflows they support

On the create-donation screen, the pickup deadline is entered as a clock time. `lib/time.ts` converts it to an absolute instant, rolling a time already passed today forward to tomorrow. The pickup pin is filled from the browser’s geolocation (*Use my location*) or typed as latitude and longitude. The screen also resizes a chosen photograph in the browser before upload, because the server caps image data at 256 KiB and a typical phone photograph is several megabytes. According to the project’s task record, a 4032×3024 photograph was reduced from about 3.46 million

data-URL characters to about 221 thousand in a browser measurement. That figure is quoted from the record, not re-measured for this report. The phone-layout screens under `/m/*` still contain demonstration residue. In particular, the camera-based create screen simulates reading a photograph with a timer and fixed text, and submits a fixed pickup address. It is *not* a computer vision feature, and is listed as open work (Section 3.3.1).

2.2.7 Security and Privacy Considerations

Table 2.16 consolidates the security and privacy controls described above against common web-application risk categories [OWA21], together with the residual risk known at mid-term.

Concern	Implemented control	Residual risk
Broken access control	Server-side role, ownership (SQL scope), lifecycle and verification checks; 404 on unauthorised ids; frontend guards not trusted	Transition route checks the state table before scope, so its error wording can reveal a donation id's current status
Privilege escalation	<code>admin</code> refused at registration; privileged fields absent from self-service schemas; role re-read per request; admin lockout guards	None known
Credential attacks	bcrypt; uniform sign-in error; per-address rate limits	Distributed attacks; registration reveals whether an email exists (409)
Session handling	Signed, expiring JWT; suspension effective next request	No revocation or refresh; token readable by page scripts; no CSP headers
Secrets	Fail-closed signing key, minimum length, public key refused	Deployment secret management not yet designed
Exposure of donor location	Coarse pickup area until a courier claims; exact pin only to donor, bound or verified organisations and admins	Photo, description and notes are not scoped
Exposure of organisation location	Blurred ranking position for other readers; directory scoped; score and distance withheld	Radius gate remains a slow membership oracle (rate-limited, not closed)
Race conditions	Conditional status writes and claim	Expiry sweep unguarded; counter increments read-then-write
Resource abuse	Rate limits; bounded text and images	Per-process counters; profile text unbounded
Unhandled errors	Explicit, human-readable errors for known cases	No global exception handler: other server errors return an empty 500

Table 2.16: Security and privacy controls and residual risks

Results and Evaluation

At the mid-term point, FoodLink has not been deployed or used by real donors, organisations or couriers. Its evaluation therefore rests on *verification*: checking that the implemented behaviour matches the requirements, and that the security, privacy and integrity properties hold under adversarial and concurrent use. This chapter describes the testing strategy, reports the results actually obtained while preparing this report, and analyses what they show, including the system’s current limitations.

3.1 Testing Methodology

Testing follows the project’s principle that a property which matters must be held by an automated test [PM15]. Every fix from the audits (Section 3.3) arrived with regression tests that fail against the code before the fix. The strategy has four parts:

- **Unit Testing:** The ranking arithmetic is tested directly, without HTTP or a database, in `test_matching_scores.py` and `test_requirement_matching.py`. These cover each criterion’s formula, the eligibility gates, unit handling, the blurring of positions and the requirement tie-break. Configuration validation and the sliding-window rate limiter are also unit-tested (`test_config.py`, `test_rate_limit.py`). In the frontend, Vitest unit tests cover the pure library modules: wire-to-domain adapters, time and urgency, distance and location display, per-account impact, the HTTP client, and the photo resizing pipeline, with its browser image functions injected because jsdom has no canvas.
- **Integration Testing:** Most backend tests exercise the complete HTTP stack through FastAPI’s test client against a fresh in-memory SQLite database per test, with no mocks (D-17). A test registers or seeds real accounts, signs in, and calls the API exactly as the frontend does. This is how authorization boundaries are tested: every role is tried against every scoped read and every transition. Concurrency tests use a file-backed database and two genuine transactions interleaved by hand, so that a lost-update race is actually reproduced and then shown to be refused. Migration tests build a database from the Alembic revision history and assert that it matches the models. Frontend component tests render screens with Testing Library in jsdom to check what each screen shows and, just as importantly, what it must not show.
- **Performance Testing:** No load, stress or response-time testing has been performed. Performance-related work so far consists of design measures (eager

loading, bounded payloads, a 500-row list cap) and one browser measurement of the photo-resizing pipeline recorded during Task 40 (Section 2.2.6).

- **User Testing:** Not yet conducted. The proposal’s pilot, with campus donors, recipient organisations and student volunteers [GSHS26], remains future work (Chapter 4).

Continuous integration. A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push to the main branch and on every pull request. It has two parallel jobs. The *backend* job, on Python 3.13, runs `pytest code/tests`, then `alembic upgrade head` and `alembic check` on a throwaway database to catch a model change committed without a migration. The *frontend* job, on Node.js 20, runs `npm ci`, `npm test` and `npm run build`; the build step includes the TypeScript compiler. The workflow deploys nothing and holds no secrets (D-25).

Table 3.1 groups the 27 backend test files by the property they protect.

Area	Backend test files (in <code>code/tests/</code>)
End-to-end paths, accounts, configuration	<code>test_api.py</code> , <code>test_auth_admin.py</code> , <code>test_config.py</code> , <code>test_rate_limit.py</code> , <code>test_donation_rate_limit.py</code> , <code>test_migrations.py</code> , <code>test_recipient_reverification.py</code>
Input validation	<code>test_donation_input_bounds.py</code> , <code>test_patch_null_semantics.py</code>
Read and disclosure scopes	<code>test_donation_reads.py</code> , <code>test_recipient_reads.py</code> , <code>test_volunteer_reads.py</code> , <code>test_requirement_reads.py</code> , <code>test_courier_pickup_disclosure.py</code>
Lifecycle and concurrency	<code>test_lifecycle_authorization.py</code> , <code>test_pickup_release.py</code> , <code>test_cancellation_reliability.py</code> , <code>test_courier_claim.py</code> , <code>test_lifecycle_concurrency.py</code> , <code>test_available_donations_deadline.py</code> , <code>test_volunteer_delivery_history.py</code> , <code>test_requirement_lifecycle.py</code>
Matching and location privacy	<code>test_matching_scores.py</code> , <code>test_match_score_consistency.py</code> , <code>test_match_distance_privacy.py</code> , <code>test_donation_privacy_scope.py</code> , <code>test_requirement_matching.py</code>

Table 3.1: Backend test suite organised by the property under test

3.2 Performance Metrics

All results in Table 3.2 were obtained by running the checks on the repository at commit `e588b35` on 15 September 2026 while preparing this report. The backend database was redirected to a temporary file so that no project file was changed.

Metric	Target	Achieved
Backend automated tests (<code>pytest code/tests</code>)	All pass	495 passed, 0 failed, 1 library deprecation warning; 337 s
Frontend automated tests (<code>npm test</code>)	All pass	156 passed, 0 failed, in 18 files
Frontend type check (<code>tsc --noEmit</code>)	No errors	No errors
Migration drift (<code>alembic upgrade head</code> , <code>alembic check</code>)	No drift	Upgraded to <code>ae4636b1e6d4</code> ; no new operations detected
Lifecycle states enforced	9	9
API routes implemented	All proposal workflows	27 routes across 5 routers plus health
Role portals	4	4 web portals, plus phone-layout screens
P1 findings of the 10 September 2026 audit	All fixed	5 of 5 fixed, each with regression tests

Table 3.2: Validation results at mid-term

The frontend production build (`npm run build`) was not re-run for this report, because it writes build output into the repository. It runs in CI on every push. No response-time or throughput figures are reported, because none have been measured.

The proposal defined outcome metrics for a pilot [GSHS26]. Table 3.3 records how far each has progressed. The system can now *compute* the ledger-based ones. None has been *measured*, because no real usage has taken place.

3.3 Analysis

- **Did the project meet its objectives?** Objectives 1 to 5 and 7 are met and verified (Table 1.2). Objective 6 is met as instrumentation but cannot yet produce meaningful numbers without real use. Objective 8 (deployment and pilot) has not started.
- **Where did the work go beyond the proposal?** The proposal described a

Proposal metric	Proposal target	Mid-term status
Median time-to-claim	≤ 45 minutes	Instrumented in GET /api/metrics; not measured
Rescue rate	$\geq 80\%$	Instrumented; not measured
Expiry-loss rate	Counter-metric	Instrumented; depends on the manual sweep having run
Handover time	Tracked	Instrumented (median acceptance to delivery); not measured
User clarity (1–5 rating)	Tracked	Not instrumented; no feedback feature exists
Service availability	$\geq 99\%$ in pilot	Not applicable yet; nothing is deployed

Table 3.3: Evaluation metrics from the proposal and their mid-term status

first iteration of posting, acceptance, courier status updates and a shared timeline, with ranking and requirements as later deliverables [GSHS26]. Both later deliverables are already implemented. So is a set of properties the proposal did not name, which emerged from the audits: requirement-aware tie-breaking, per-reader disclosure scopes for donor and organisation locations, race-safe conditional writes for every transition, abuse limits, and reliability accounting that is not distorted by cancellations.

- **What challenges were encountered?** The hardest problems were not features but properties that only appear under adversarial or concurrent use. Each was found by an audit and reproduced against the running application before it was fixed. Table 3.4 summarises the most significant.
- **How were they resolved?** Each fix was the smallest change that made the property hold everywhere rather than only on the reported path. It was recorded as a decision with its reasoning and protected by tests that fail without it. None of the five P1 fixes required a database schema change.

3.3.1 Current Limitations

The following limitations are verified against the repository and are stated so that the system is not described as more complete than it is.

1. **No deployment.** Nothing is hosted. There is no deployment configuration, and PostgreSQL, although supported in principle, is untested and has no driver pinned.

Challenge (audit finding)	Resolution	Evidence
Newly self-registered organisation and courier accounts could read every open donation's exact pickup pin, address and donor name (P1-1)	Open pool restricted to verified organisations (D-53); couriers see a coarse area until they claim (D-57)	<code>test_donation_reads.py</code> , <code>test_courier_pickup_disclosure.py</code>
Verification survived an organisation renaming or moving itself (P1-2)	A real change to name or coordinates clears verification in the same commit (D-54)	<code>test_recipient_reverification.py</code>
Two organisations accepting at the same moment both succeeded, even on SQLite (P1-3)	Every status write made conditional on the expected state; the loser receives 409 and its side effects roll back (D-55)	<code>test_lifecycle_concurrency.py</code>
Donor cancellations lowered the organisation's reliability; donors could cancel collected food (P1-4)	Cancellation withdraws the counted acceptance; donors cannot cancel after pickup (D-58)	<code>test_cancellation_reliability.py</code>
Ordinary phone photos made donation posting fail against the image cap (P1-5)	Browser-side resize and JPEG re-encode before upload (D-56)	<code>lib/__tests__/image.test.ts</code>
Ranking results revealed exact distances, from which organisation addresses could be triangulated (HA-3)	Organisation positions blurred to a 0.01° grid for other readers; distances and frozen scores withheld (D-45, D-47)	<code>test_match_distance_privacy.py</code>
Unlimited donation posting allowed spam and probing of the radius gate (P2-1)	Per-account and per-address limits (D-59)	<code>test_donation_rate_limit.py</code>
Two quantity criteria were mathematically the same criterion counted twice, and ignored units (HA-4, HA-5)	Capacity headroom redefined in absolute meals; non-meal units not assessed (D-42)	<code>test_matching_scores.py</code>

Table 3.4: Principal challenges found by audit and how they were resolved

2. **No notifications.** Participants learn about new donations and status changes only by opening the portal.
3. **Missing web controls.** Cancellation, pickup release and account management work through the API but have no portal screen. Neither the registration form nor the organisation profile screen collects map coordinates. An organisation created through the portal therefore cannot be ranked, since coordinates are an eligibility gate, until its pin is set through `PATCH /api/recipients/me`. The same applies to organisations created by an administrator. In practice, only organisations with coordinates, such as the seeded demonstration organisations, appear in rankings today.
4. **Manual expiry.** The expiry sweep must be triggered by an administrator. Its bulk write does not use the conditional guard that every other status write uses. An administrator's `ACCEPTED` $\rightarrow$ `EXPIRED` leaves the acceptance counted.
5. **Session security.** No token revocation, refresh token, multi-factor authentication or content security policy. Rate-limit counters are per process and reset on restart.
6. **No maps.** Distance is great-circle, and travel time is a flat 20 km/h estimate; there is no road routing. The pickup pin is entered through browser geolocation or typed coordinates, with no map picker.
7. **Heuristic weights.** The matching weights and the reliability prior are reasoned judgements, not values tuned on outcome data.
8. **Inline photos.** Photos are stored in the donation row and returned with every list, and every write re-downloads up to 500 donations.
9. **Scale.** Recipients are loaded and filtered in Python for each ranking. Metrics are computed in Python over every donation. SQLite allows a single writer.
10. **Validation gaps.** Account and organisation profile text is unbounded. An unexpected server error returns an empty 500 response, which the frontend presents as a connection problem.
11. **Demonstration residue.** The phone-layout camera screen simulates reading a photo and submits a fixed address, and one mobile profile screen shows a hard-coded email. The `/ngo/available/:id` deep link does not select the donation.
12. **Metric caveats.** The completed-meals total adds quantities across different units. The expiry-loss rate is accurate only after the sweep has run.

Feasibility Report

A feasibility study asks whether a system can be built and used successfully under real technical, operational, economic, schedule and legal constraints [Som16; PM15]. At proposal stage, feasibility can only be argued. At the mid-term point, much of FoodLink exists, so this chapter assesses feasibility against evidence from the implementation. It distinguishes throughout between three categories:

- what is **already implemented** and verified;
- what is **feasible but not implemented**, meaning the design accommodates it and no technical obstacle is known;
- what remains **future work** requiring further design or decisions.

4.1 Technical Feasibility

4.1.1 Technology Stack

Every component of the stack is mature, widely used and well documented (Table 2.2): Python with FastAPI, SQLAlchemy, Alembic and Pydantic on the server; React, TypeScript and Vite in the browser; pytest and Vitest for testing [Ram26; SQL26b; SQL26a; Pyd26; Met26; Vit26a; pyt26; Vit26b]. The project needed no experimental technology, no proprietary component and no external service. The strongest evidence of technical feasibility is that the complete workflow from posting to completion runs today and passes 651 automated tests (Section 3.2).

4.1.2 Frontend

Implemented. Four role portals are built with standard React patterns: context providers, a router and a single HTTP client. The browser performs only presentation work, plus one computation that could not live on the server, namely resizing a photograph that only the browser holds. Frontend feasibility risks are therefore low. *Feasible but not implemented.* Routing phone visitors to the existing `/m/*` layouts automatically (a detection hook exists but is unused), generating the TypeScript API types from the server’s OpenAPI document instead of maintaining them by hand, and adding controls for the API-only operations. *Future work.* Code-splitting per portal and a caching data layer, if profiling shows the full-list reloads to be too slow.

4.1.3 Backend and APIs

Implemented. 27 REST routes cover every proposal workflow. The API is stateless apart from rate-limit counters, documents itself through OpenAPI, and expresses its rules as inspectable data: transition, role and scope tables. Complexity was concentrated where it is justified, in authorization scopes, conditional writes and disclosure rules, rather than in infrastructure. *Feasible but not implemented.* A global exception handler that returns a readable message and a correlation id; bounds on profile text fields; a scheduled caller for the expiry sweep. The sweep is already an idempotent endpoint, so a cron job holding an administrator token would suffice.

4.1.4 Database

Implemented. Six normalised tables under Alembic migration control. Timestamps are normalised to UTC. SQLite needs no database server, which kept development setup to installing packages and setting one environment variable. *Feasible but not implemented.* Moving to PostgreSQL. All data access goes through SQLAlchemy, and the lifecycle's concurrency guards are plain conditional UPDATE statements that need no engine-specific feature. However, no PostgreSQL driver is pinned, the suite has never run against PostgreSQL, and the guards' behaviour there depends on PostgreSQL's default READ COMMITTED isolation level (D-28). Enabling foreign-key enforcement in SQLite is a small, localised change. *Constraint.* SQLite permits one writer at a time [SQL26c]. It is suitable for a single-process pilot but not for several API worker processes.

4.1.5 Authentication

Implemented. Password hashing with bcrypt, signed expiring tokens, a fail-closed signing key, per-request re-reading of the account, and administrator-only creation of administrators. *Feasible but not implemented.* Token revocation (for example a version number on the user row checked on each request), shorter-lived access tokens with refresh, and security headers. *Future work.* Email verification, multi-factor authentication and an administrator audit log, which need product decisions and, for email, an external service.

4.1.6 Deployment Considerations

Not implemented: there is no deployment configuration of any kind. Hosting FoodLink is technically straightforward, since it is one ASGI application and one static bundle. The implementation, however, fixes several decisions that a deployment must respect:

1. migrations currently run inside the application's start-up, which is safe only while exactly one process runs (D-23);
2. rate-limit counters live in process memory, so several processes would multiply the effective limits unless a shared store is introduced (D-27);
3. behind a reverse proxy, the server must be configured to trust the proxy's forwarded client address, or all users will share one rate-limit budget;
4. `CORS_ORIGINS` must list the deployed frontend's origin, and a real signing key must be supplied as a secret;
5. the expiry sweep needs an external scheduler.

A single-process deployment with SQLite is feasible now. A multi-process deployment is feasible once items 1 and 2 are addressed and PostgreSQL is validated.

4.1.7 Implementation Complexity

The domain is small (six tables, nine states), but the implementation is not trivial. Much of the effort went into properties that tutorials rarely cover: per-role read scopes expressed as SQL, disclosure rules that change what a record says to different readers, race-safe transitions, and privacy protections against location inference. The project has shown that a four-person student team can deliver this complexity by working in small, tested increments with written decisions. The remaining work (Section 4.4) is of lower algorithmic complexity than what has already been built.

Assessment: technically feasible, and already demonstrated for the core system. The main open technical risk is an untested production database and deployment topology.

4.2 Operational Feasibility

Operational feasibility asks whether the intended users can and will use the system as designed.

Donors. A donor signs up, opens *Create Food Donation*, fills in one form and submits it. The photo is optional and resized automatically. The deadline is entered as a clock time, and a time that has already passed today is taken to mean tomorrow. The pickup pin comes from the browser's location service or is typed as latitude and longitude; there is no map picker, which is a usability gap for non-technical donors. The donor can then follow the donation's timeline and browse the needs board to see what verified organisations want. The flow suits a kitchen worker posting during

clean-up, as the proposal required. Two operational gaps remain. A donor must re-open the portal to see progress, since nothing is pushed. A donor who needs to withdraw a donation has no button for it yet.

Recipient organisations. An organisation registers and then waits for an administrator's verification. Until verified, it sees an explanation rather than an empty screen. Once verified, its *Available Donations* screen lists collectable donations, each with its own score and plain-language reasons, and acceptance is a single action. It confirms receipt from *Accepted Deliveries* and maintains standing needs on *Food Requirements & Demand Signals*. Verification adds a deliberate human delay before an organisation becomes useful. That is the cost of the trust model, and an administrator must actually carry out the vetting. A more serious operational gap is that no screen lets an organisation set its map coordinates. A newly registered organisation can browse and accept once verified, but it is never *suggested* to donors until its pin is set through the API. Adding coordinate fields (or a location button like the donor's) to the organisation profile is a small, high-priority change.

Volunteer couriers. A courier sees unclaimed pickups with a coarse area, enough to judge whether a run is practical, and claims one. The exact address and donor appear only after the claim. The courier then records pickup and delivery with one button each, and a history page lists completed work. The availability toggle is advisory: it is recorded but does not currently gate claiming. If a courier cannot complete a claimed run, only the accepting organisation (through the API) or an administrator can release it at present.

Administrators. Administration involves verifying organisations, monitoring donations, running the expiry sweep and reviewing analytics. These tasks are supported by portal screens. Account suspension and administrator creation are available only through the API or the command-line tool, which is workable for a small pilot run by technical staff but not for a larger service.

Practicality of the workflow. The workflow keeps a human decision at every point of custody. An organisation must accept, a courier must claim, and the organisation must confirm receipt. The system never assigns food automatically, which matches how the proposal expected real handovers to work [GSHS26]. The main operational risk is *timeliness without notifications*. The platform removes the need to find a recipient, but it cannot yet tell that recipient that food is waiting.

Assessment: operationally feasible for a supervised pilot on one campus. Notifications, portal controls for cancellation and release, and an account-management screen are needed before unsupervised use.

4.3 Economic Feasibility

This section identifies cost drivers without inventing monetary figures, since no hosting provider, pilot size or service contract has been chosen.

Development resources. Development effort is the time of the four-member student team within the course. No paid developers, contractors or consultants are involved.

Software and tooling. Every runtime dependency and development tool in the repository is open-source software, available without licence fees. The only services used are a Git hosting platform (for the repository, CI runs and the course documentation pages) and an online LaTeX editor for reports. The report's cost assumptions do not depend on any paid tier of either.

Infrastructure. There is currently no infrastructure cost, because nothing is deployed. The backend makes no calls to paid external APIs (no maps, SMS or email providers), so operating costs would be limited to hosting. A pilot would need, at minimum, one small application server and, later, a managed PostgreSQL database and static hosting for the frontend. Features on the roadmap would add costs. Email or SMS notifications carry per-message charges; object storage for photos and road-distance routing typically carry usage-based charges. Their magnitude depends on provider and volume and has not been estimated.

Cost-reducing design choices already made. SQLite by default avoids a database server during development and a small pilot. Straight-line distance avoids a routing API. Browser-side photo resizing reduces storage and bandwidth. No paid third-party service is integrated.

Benefit side. The proposal's intended benefit is surplus food reaching people instead of being discarded [GSHS26]. The platform can now measure this through rescue rate and expiry-loss rate once it is used. No benefit figures are claimed here, because none have been measured.

Assessment: economically feasible at course and pilot scale, with near-zero software cost. Hosting and messaging costs for a wider service remain to be estimated once a provider and scale are chosen.

4.4 Schedule Feasibility

Course timeline. The course schedules a proposal in week 4, this prototype-stage report in week 7, and the final report and seminar in week 17 [Ven26].

Progress so far. The repository’s first commit is dated 29 July 2026. By commit `e588b35` (13 September 2026) it contained 85 commits. The backend and frontend were built, mock data was removed, and two audit cycles were run and their P1 findings fixed. Table 4.1 summarises implementation maturity by area.

Area	Maturity	Remaining
Donation lifecycle	Complete, hardened	Guard the expiry sweep; admin expiry accounting
Matching and explainability	Complete	Tuning weights needs outcome data
Authentication and authorization	Complete for pilot	Revocation, security headers, profile bounds
Requirements and needs board	Complete	Structured food categories (optional)
Web portals	Complete for core flows	Cancel, release and account-management controls; deep-link fix
Phone layouts	Partial	Remove demonstration residue; entry routing
Testing and CI	Strong backend; partial frontend	Frontend tests for data context, actions and forms
Deployment	Not started	Configuration, PostgreSQL validation, scheduler, secrets
Pilot and measurement	Not started	Recruit users; collect ledger metrics

Table 4.1: Implementation maturity and remaining work by area

Remaining work before the final evaluation. In priority order, as recorded in the project task list: the remaining P2 items (a global exception handler, profile field bounds, the deep-link bug, frontend test gaps and mobile residue); portal controls for the API-only operations; a deployment configuration with a scheduled expiry sweep; and, if deployment succeeds, a short pilot to produce real values for the proposal’s metrics. Each item is small or medium in size compared with the work already completed.

Realistic future development areas. Beyond the course timeline: notifications, road distances, object storage for photos, weight tuning from outcome data, and broader organisation onboarding.

Assessment: schedule-feasible. The core system was delivered by mid-term. The remaining course-scope work is bounded. The principal schedule risk is the deployment and pilot, which depend on arranging real participants as well as on engineering.

4.5 Legal, Security and Privacy Feasibility

FoodLink stores personal data about identifiable people: names, email addresses, phone numbers, organisation addresses, donors' pickup locations (which may be homes), photographs and activity history. Any real deployment in India would have to comply with the Digital Personal Data Protection Act, 2023 [Min23]. This report does not offer a legal opinion. It records what the implementation already does that supports compliance, and what is missing.

Authentication. *Implemented:* hashed passwords, uniform sign-in errors, rate-limited credential endpoints, and immediate effect of suspension. *Missing:* revocation of individual sessions, multi-factor authentication and email verification.

Authorization. *Implemented:* server-side enforcement of role, ownership, lifecycle and trust on every request, with scoping inside database queries and no reliance on client-side checks. Automated tests cover every role against every scoped read and transition.

Protection of sensitive pickup information. *Implemented:* a donor's exact pin, address text and name are shown only to the donor, organisations able to act on the donation, administrators, and the one courier who has claimed the run. Other couriers see a roughly one-kilometre area. Organisation positions are blurred in rankings read by others, and organisation and courier phone numbers are scoped to readers with a working relationship. *Missing:* the free-text description, photo and notes are not scoped, so a donor who types an address into the description would expose it.

Verification and trust controls. *Implemented:* an organisation cannot be ranked, see the open pool or accept food until an administrator verifies it, and loses verification if it changes its name or location. *Operational requirement:* the legal and reputational value of this control depends on administrators performing genuine vetting, which is a process outside the software.

Data minimisation and user rights. *Implemented:* registration asks only for what each role needs; a courier’s coordinates are write-only; and the event ledger records server facts, not free-form surveillance. *Missing and required before public use:* a privacy notice and consent at sign-up; a way for users to obtain, correct or delete their data (accounts can currently only be deactivated); a defined retention period for donations, photos and events; and hosting and backup arrangements that protect stored data.

Food safety and liability. The platform records the storage type, optional preparation time and deadline so that a recipient can decide, and it does not certify food. The proposal places inspection with the recipient at handover [GSHS26]. The current interface does not yet present terms of use stating this division of responsibility. That would be needed, together with review of any food-safety obligations applicable to food donation, before public operation.

Assessment: the security and privacy architecture is already stronger than a typical prototype and suitable for a supervised pilot with informed participants. Public operation additionally requires the privacy, consent, data-rights, retention and terms-of-use measures listed above.

4.6 Feasibility Summary

Overall, FoodLink is feasible. Its core has moved from proposal to working, tested software. What remains is predominantly deployment, operational completeness and legal readiness for real users, rather than open technical questions.

Dimension	Implemented	Feasible, not implemented	Future work	Verdict
Technical	Full workflow, 27 routes, 6 tables, 651 passing tests, CI	PostgreSQL, scheduler, exception handler, token revocation	Multi-process deployment, routing, object storage	Feasible
Operational	Four portals covering every core step	Cancel, release and account screens; mobile entry routing	Notifications; broader onboarding	Feasible for pilot
Economic	Open-source stack; no paid services	Single-server pilot hosting	Messaging and storage costs	Feasible
Schedule	Core system by mid-term	P2 fixes, portal controls, deployment configuration	Pilot, metric collection	Feasible
Legal, security, privacy	Scoped access, location protection, verification	Security headers, profile bounds	Privacy notice, consent, data rights, retention, terms	Conditional

Table 4.2: Summary of feasibility by dimension

Conclusion

5.1 Summary of Work

This report has presented FoodLink at its mid-term stage. The project set out to shorten the time between surplus food being posted and a trustworthy organisation accepting it, and to see the food through collection and delivery with a record of every step (Section 1.1.1).

By mid-term, the team has built and verified the core system, meeting Objectives 1 to 5 and 7:

- a FastAPI backend with 27 routes over a six-table schema managed by Alembic, and a React frontend with four role portals;
- the complete nine-state donation lifecycle. Transitions are declared as data, checked for role, ownership and legality, and committed through conditional writes that are safe under concurrent use. Every transition is recorded in an append-only, server-stamped ledger;
- an explainable matching engine: five published, weighted criteria behind three eligibility gates, with per-criterion scores and plain-language reasons, and requirement-aware tie-breaking that never alters a score;
- an access model of role, ownership, lifecycle and administrator verification, with per-reader scopes that protect donors' pickup locations and organisations' positions;
- standing requirements and a donor needs board, ledger-derived platform metrics, and administrator tools for verification and expiry;
- 495 backend and 156 frontend automated tests, all passing, run in continuous integration together with a migration drift check.

Objective 6 is met as instrumentation, awaiting real usage. Objective 8, deployment and a pilot, has not started.

5.2 Key Findings

- **Finding 1:** *A transparent heuristic is sufficient and verifiable.* Every ranking can be recomputed by hand from published formulas (Table 2.13), and explanations are generated from the same numbers. Tying the explanations to the

scoring code in this way exposed and removed a real defect: two criteria were secretly the same criterion counted twice.

- **Finding 2:** *In a multi-party workflow, most of the difficulty lies in access and integrity rather than in features.* The most serious problems found were not missing screens. They were self-registered accounts able to read donors' addresses, verification that survived a change of identity, and two organisations able to accept the same donation simultaneously. Each was invisible in ordinary single-user testing.
- **Finding 3:** *Race conditions are real even on SQLite.* A lost update between two acceptances was reproduced on the project's default database. Making the database evaluate the precondition as part of the write, rather than locking or checking in application code, fixed it in a way that is testable on the engine in use.
- **Finding 4:** *Privacy needs reader-specific answers.* Hiding records was not enough. What a record *says* must depend on who reads it, as with the coarse pickup area shown until a claim and the blurred organisation positions, because figures such as distances and scores can be used to infer the locations they are derived from.
- **Finding 5:** *Evidence-based metrics depend on design decisions made early.* Choosing an append-only event ledger at the start means time-to-claim and rescue rate can be computed from server facts. Only real usage is now needed to produce them.

5.3 Future Work and Recommendations

The following items are planned or recommended. None is implemented today. They are listed in the order the team recommends, from course-scope work to longer-term development.

1. **Close the remaining audit items:** a global exception handler returning a readable message; length bounds on profile fields; guarding the expiry sweep's status write; fixing the organisation deep link; removing the demonstration residue from the phone layouts; and frontend tests for the data context, actions and donation form.
2. **Complete the portal:** a map-pin field on the organisation profile (without which a portal-registered organisation cannot be ranked), and controls for cancelling a donation, releasing a pickup and managing accounts, all of which the API already supports.

3. **Deploy a pilot:** a deployment configuration, a real signing key held as a secret, a scheduled expiry sweep, migration of start-up tasks out of the application process, and validation on PostgreSQL if more than one process is needed.
4. **Run the pilot and measure:** recruit campus donors, organisations and couriers, and report the proposal's time-to-claim, rescue-rate and handover metrics from the event ledger.
5. **Timeliness:** in-app, email or SMS notifications when a donation is posted near an organisation or changes state.
6. **Security hardening:** token revocation, security headers, dependency auditing in CI, and an administrator audit log.
7. **Usability and scale:** a map-based pin picker, object storage for photographs, pagination, and aggregation of metrics and eligibility filtering in the database.
8. **Matching refinements:** structured food categories on requirements so that food type can be compared as well as quantity; road distance; and, once outcome data exists, tuning of the weights. Any learned ranker would replace `score_pair` behind the existing interface. It would need to preserve explainability, and would be justified only by measured improvement.
9. **Legal readiness:** a privacy notice and consent, data-access and deletion requests, retention rules, and terms of use stating that recipients inspect food at handover (Section 4.5).

5.4 Final Remarks

FoodLink began as a proposal for a coordination tool and is now a working, tested system whose rules can be read, whose rankings can be checked by hand, and whose history cannot be rewritten by its users. The most valuable lessons came from treating the system adversarially: asking what a newly registered stranger could read, what happens when two people act at the same instant, and whose record a cancellation should count against. The remaining work before the final evaluation is to take the system to real users and let the event ledger measure whether it achieves its purpose of getting surplus food to people before it expires.

System Design: UML Diagrams

This chapter presents FoodLink’s design as Unified Modeling Language (UML) artefacts [Obj17; BRJ05]: a use case diagram with detailed use case scenarios, a class diagram and an activity diagram. Every element was derived from the repository at commit e588b35. Actors are values of `UserRole`; use cases are implemented routes and screens; classes are the ORM models and the matching module; and activity branches are decisions the server actually takes. Earlier copies of these diagrams in `docs/uml/` predate several decisions described in Chapter 2 (D-50, D-52, D-53, D-55, D-57 and D-58). The versions below are corrected redraws, with their TikZ sources in the report’s `diagrams/` directory.

6.1 Use Case Model

6.1.1 Use Case Diagram

Figure 6.1 shows the four role actors and the 26 use cases they drive.

Actors. *Donor*, *Recipient Organisation (NGO)*, *Volunteer Courier* and *Admin* correspond exactly to the four values of `UserRole`. They specialise an abstract *Platform User*, which carries the use cases common to every signed-in role: signing in, managing one’s own account and password, and tracking the status timeline of the donations one may read. *Register Account* is attached to the abstract actor for layout reasons, but only the donor, organisation and courier roles may register. Administrators come from the command line or from another administrator (*Manage User Accounts*).

Relationships. *Post Donation*, *View Ranked Matches*, *Browse Open Donations* and *Accept Donation* each `<<include>>` the internal use case *Rank Recipient Organisations*. Ranking always runs at posting, for each open donation an organisation reads, whenever matches are requested, and once more at first acceptance to freeze the accepting organisation’s score. *Review Match Explanation* `<<extend>>`s *Browse Open Donations* when the organisation selects a donation that is still open and for which it is eligible.

Conditions not drawn as relationships. Browsing the open pool and accepting require an administrator-verified organisation. Acceptance is refused once the deadline has passed. *Cancel Donation* is available to the donor only before pickup. *Override Lifecycle Transition* represents the administrator’s presence in every entry of

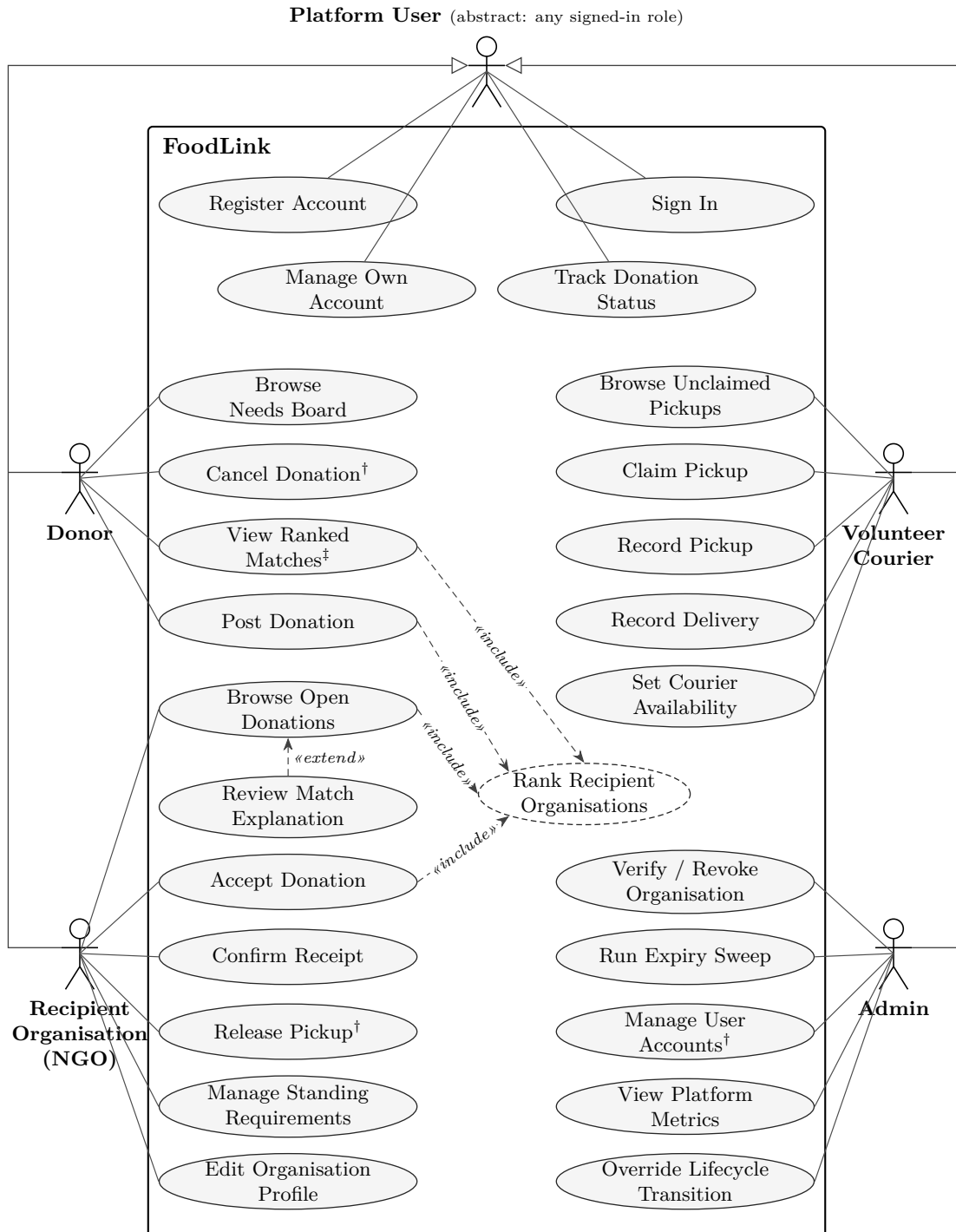


Figure 6.1: Use case diagram of FoodLink. †: implemented and tested in the API, not yet offered in the web portal. ‡: API and phone-layout camera screen only.

TRANSITION_ROLES: an administrator may accept on an organisation's behalf, record pickup, delivery or completion, cancel (including after pickup) and expire donations. The claim is the exception, since an administrator has no courier profile.

6.1.2 Use Case Scenarios

The scenarios below describe the main use cases in a structured form. Each alternative flow names the HTTP status and, where useful, the message the server returns. Routes are relative to the API root; screens are the web portal's.

UC-01 — Register Account	
Primary actor	Prospective donor, recipient organisation or courier (not signed in)
Goal	Obtain an account in a self-service role and reach that role's portal.
Preconditions	The email address is not registered. Fewer than 10 registrations have come from the visitor's network in the past hour.
Trigger	The visitor chooses to create an account on the sign-in page.
Main success flow	1. The visitor selects a role tile (Donor, Recipient or Volunteer) and enters name, optional organisation name, email and password. 2. The client sends <code>POST /api/auth/register</code>. 3. The server validates the body: a valid email, a password of 8–128 characters, and a role among <code>SELF_SIGNUP_ROLES</code>. 4. The server lower-cases the email, confirms it is unused, stores a bcrypt hash and creates the user row. 5. For a courier, it creates a Volunteer profile. For an organisation, it creates a Recipient profile that is <i>unverified</i>, named after the organisation (or the person), with capacity 100. 6. It commits and returns 201 with an access token and the account. The client stores the token and opens the role's portal.
Alternative and exception flows	• Email already registered: 409, "An account with that email already exists". • Role <code>admin</code> sent: 422, "Administrator accounts cannot be created through registration". • Malformed email or short password: 422 with field errors. • Over the registration limit: 429 with Retry-After.
Postconditions	An active account exists. A new organisation cannot be ranked, see the open pool or accept until an administrator verifies it (UC-13). It also cannot be ranked until coordinates are set, which the API accepts at registration but the portal form does not collect.
Business rules	D-04: administrators are never self-registered. A duplicate email is reported distinctly, which reveals that an account exists (known limitation).

Table 6.1: Use case scenario UC-01: Register Account

UC-02 — Sign In	
Primary actor	Any registered user (donor, organisation, courier, administrator)
Goal	Start an authenticated session in the correct portal.
Preconditions	The account exists. Fewer than 30 sign-in attempts from the network in the past five minutes.
Trigger	The user submits email and password on <code>/login</code> .
Main success flow	1. The client sends <code>POST /api/auth/login</code> as an OAuth 2.0 password form (email in <code>username</code>). 2. The server finds the user by lower-cased email and verifies the bcrypt hash. 3. It confirms the account is active and returns a JWT (HS256, 720-minute expiry) with the account. 4. The client stores the token and navigates to the account's own portal, whatever tile was highlighted. 5. On every later request, the server verifies the token and re-reads the user row.
Alternative and exception flows	• Unknown email or wrong password: 401, "Incorrect email or password" (the same message for both). • Suspended account: 403, "This account has been deactivated. Contact a platform administrator." • Over the sign-in limit: 429 with Retry-After. • Token expired, or account suspended mid-session: the next request returns 401, and the client signs the user out everywhere.
Postconditions	The user is signed in until the token expires or they sign out. Sign-out is client-side; the token is not revoked on the server.
Business rules	The token's role claim is never trusted; the stored role decides (D-03).

Table 6.2: Use case scenario UC-02: Sign In

UC-03 — Post Donation	
Primary actor	Donor (an administrator may also post)
Goal	Offer surplus food to nearby verified organisations before it expires.
Preconditions	Signed in as a donor. Under 10 posts in the past hour for the account and under 30 for the network.
Trigger	The donor submits the <i>Create Food Donation</i> form.
Main success flow	1. The donor enters food details, pickup address and pin (<i>Use my location</i> or typed coordinates), an optional photo and preparation time, and the deadline as a clock time. 2. The browser resizes the photo (at most 1280 px, JPEG, within 256 KiB) and converts the deadline to the next future instant. 3. <code>POST /api/donations</code>: the server checks role and rate limits, then field bounds, coordinates and image format. 4. It confirms the deadline is in the future, stores the donation as <code>AVAILABLE</code> and appends that event. 5. It ranks every organisation. Verified organisations with coordinates within 8 km are scored; active requirements break ties. 6. If any is eligible, it freezes the top score in <code>match_score</code> and records <code>MATCHED</code> (“Top match: <i>name</i>”). 7. It commits once and returns 201. The donation appears in <i>My Donations</i> with its timeline.
Alternative and exception flows	• No eligible organisation: the donation stays <code>AVAILABLE</code> (still a success). • Deadline not in the future: 422, “Pickup deadline is already in the past”. • Invalid or over-long fields, or <code>imageUrl</code> neither an image data URL nor an http(s) link: 422. • Undecodable or incompressible photo: refused in the browser with a message. • Account or network budget exhausted: 429, naming the limit. Organisation or courier role: 403.
Postconditions	The donation is open to every verified organisation until its deadline, and every transition from here on is recorded.
Business rules	Nothing is assigned at posting. The frozen score and distances are not shown to the donor (D-47).

Table 6.3: Use case scenario UC-03: Post Donation

UC-04 — View Ranked Matches and Match Explanation	
Primary actor	Recipient organisation. Secondary: donor and administrator (ranked list through the API)
Goal	Understand how well a donation suits an organisation, and why.
Preconditions	The organisation is verified, and the donation is open (AVAILABLE or MATCHED) with its deadline not passed. For the ranked list, the caller can read the donation.
Trigger	The organisation selects a donation on <i>Available Donations</i> ; or a client calls <code>GET /api/donations/{id}/matches</code> .
Main success flow	1. The client loads <code>GET /api/donations</code>. The server includes, for each open donation, <code>viewerMatch</code>: the organisation's own live result from <code>score_pair</code>, using its true coordinates and its own active requirements. 2. The organisation selects a donation. The match panel shows the overall score, the five criterion scores, the straight-line distance and the reason lines. 3. <i>Variant (ranked list)</i>: <code>/matches</code> returns up to <code>limit</code> (at most 25) eligible organisations in ranking order. Each has a score, sub-scores and reasons.
Alternative and exception flows	• Organisation unverified: the pool is not returned, and the screen explains that verification is pending. • Organisation ineligible (no coordinates or beyond 8 km): <code>viewerMatch</code> is null, and no breakdown is available. • Deadline passes: the donation leaves the pool on the next load. • Caller is not the organisation described (donor, courier, other organisation): its position is blurred to a 0.01° grid, <code>distanceKm</code> is null, and distance reasons omit the figure. • Donation not readable by the caller: 404.
Postconditions	Read-only; nothing is stored. Scores move with the clock because deadline comfort decays.
Business rules	Requirements affect ordering and reasons only, and only for readers entitled to them (D-52). The frozen score and the live score are different quantities (D-30).

Table 6.4: Use case scenario UC-04: View Ranked Matches and Match Explanation

UC-05 — Accept Donation	
Primary actor	Recipient organisation (an administrator may accept on its behalf)
Goal	Take responsibility for an offered donation.
Preconditions	The organisation is verified. The donation is AVAILABLE or MATCHED and its deadline has not passed.
Trigger	The organisation presses <i>Accept Donation on Available Donations</i> .
Main success flow	1. The client sends <code>POST /api/donations/{id}/status</code> with <code>status: ACCEPTED</code> and its own <code>recipientId</code>. 2. The server checks that the move is legal and that the role is organisation or administrator. 3. It refuses an overdue donation, then resolves the caller's own organisation and confirms it is verified. 4. It binds <code>recipient_id</code>, adds one to <code>accepted_donations</code>, and freezes <code>match_score</code> with this organisation's score. 5. The conditional status write succeeds. An ACCEPTED event is appended, and the transaction commits. 6. The donation moves to <i>Accepted Deliveries</i> and appears in every courier's unclaimed queue.
Alternative and exception flows	• Deadline passed: 409, "This donation's pickup deadline has passed, so it can no longer be accepted". • Organisation unverified: 403, "This organisation is awaiting verification and cannot accept donations yet". • Another organisation accepted first: 409, "Cannot move a donation from ACCEPTED to ACCEPTED"; none of this request's side effects persist. • <code>recipientId</code> names a different organisation: 403. • Account has no organisation profile: 422. • Administrator variant: names the organisation, which must also be verified.
Postconditions	Exactly one organisation is bound, with exactly one counted acceptance.
Business rules	D-06 (verification enforced at acceptance as well as ranking), D-50 (deadline), D-55 (conditional write).

Table 6.5: Use case scenario UC-05: Accept Donation

UC-06 — Claim Pickup	
Primary actor	Volunteer courier
Goal	Take on the collection and delivery of an accepted donation.
Preconditions	The caller has a courier profile. The donation is ACCEPTED and unclaimed.
Trigger	The courier presses <i>Accept Pickup</i> on a task card on the <i>Pickup Tasks</i> page.
Main success flow	1. The courier browses unclaimed pickups. Each shows food, quantity, deadline, the receiving organisation and a coarse pickupArea (for example “Approx. 30.36N, 76.37E”), with no exact pin, address or donor name. 2. The client sends <code>POST /api/donations/{id}/status</code> with status: VOLUNTEER_ASSIGNED. 3. The server checks legality and role, and finds the caller’s courier profile. 4. A conditional UPDATE sets volunteer_id only if the donation is still ACCEPTED and unclaimed. It succeeds. 5. The conditional status write succeeds, a VOLUNTEER_ASSIGNED event is appended, and the transaction commits. 6. The response and task card now show the exact pickup pin, address and donor.
Alternative and exception flows	• Another courier won the claim: 409, “Another courier has already claimed this pickup”. • Donation no longer ACCEPTED (cancelled, or already claimed by this courier’s own earlier request): 409, “Cannot move a donation from ...”. • No courier profile: 422, “No courier profile for this account”.
Postconditions	Exactly one courier is bound; other couriers no longer see the pickup.
Business rules	D-28 (atomic claim), D-57 (disclosure on claim). The availability toggle does not gate claiming. Couriers are not verified.

Table 6.6: Use case scenario UC-06: Claim Pickup

UC-07 — Record Pickup and Delivery	
Primary actor	Volunteer courier (the assigned one)
Goal	Record that the food was collected, then delivered.
Preconditions	The donation is VOLUNTEER_ASSIGNED to the caller (for pickup) or PICKED_UP by the caller (for delivery).
Trigger	The courier presses <i>Mark Picked Up</i> , and later <i>Mark Delivered</i> , on the task card.
Main success flow	1. The client sends the status update PICKED_UP. 2. The server checks legality and role, then re-reads the donation through the courier's read scope, which contains only the courier's own runs. 3. The conditional write and PICKED_UP event are committed. 4. After handing over the food, the courier sends DELIVERED, processed the same way.
Alternative and exception flows	• A different courier's run: 404 (indistinguishable from a missing donation). • Out of order (for example DELIVERED before PICKED_UP): 409. • After pickup, the donor can no longer cancel (409), and the organisation can no longer release; only an administrator can cancel.
Postconditions	The donation awaits confirmation by the organisation. The courier's history lists it from DELIVERED (D-51).
Business rules	Ownership re-checked through the read scope (D-34). Delivery counts are credited only on completion (UC-08).

Table 6.7: Use case scenario UC-07: Record Pickup and Delivery

UC-08 — Confirm Receipt	
Primary actor	Recipient organisation (the accepting one)
Goal	Close the donation as successfully received.
Preconditions	The donation is DELIVERED and bound to the caller's organisation.
Trigger	The organisation presses <i>Confirm receipt</i> on <i>Accepted Deliveries</i> .
Main success flow	1. The client sends the status update COMPLETED. 2. The server checks legality, role and ownership through the organisation's read scope. 3. It adds one to the organisation's completed_donations and the courier's completed_deliveries. 4. The conditional write and COMPLETED event are committed.
Alternative and exception flows	• Another organisation's donation: 404. • Not yet delivered: 409. • Donor or courier attempts it: 403.
Postconditions	Terminal state. The organisation's reliability score reflects the completion in future rankings; the donation counts towards the rescue rate if completed by its deadline.
Business rules	Only the organisation that received the food can confirm it.

Table 6.8: Use case scenario UC-08: Confirm Receipt

UC-09 — Cancel Donation (API only)	
Primary actor	Donor. Secondary: administrator
Goal	Withdraw a donation that will not be handed over.
Preconditions	The donor owns the donation, and it is AVAILABLE, MATCHED, ACCEPTED or VOLUNTEER_ASSIGNED. An administrator may also cancel from PICKED_UP.
Trigger	A call to POST /api/donations/{id}/status with status: CANCELLED and an optional note. There is no portal button yet.
Main success flow	1. The server checks legality, role (donor or administrator) and ownership. 2. For a donor, it confirms the donation has not been picked up. 3. The conditional status write and CANCELLED event are applied. 4. If an organisation was bound, one counted acceptance is withdrawn, never below zero, in the same transaction. 5. The transaction commits.
Alternative and exception flows	• Donor cancels after pickup: 409. • DELIVERED or COMPLETED: 409 for everyone. • Another donor's donation: 404. • A courier's pickup commits first: the cancellation is refused with 409, and nothing is withdrawn.
Postconditions	Terminal state. The organisation's reliability is unaffected.
Business rules	D-58: a cancellation is never the organisation's failure. No cancellation reason is recorded.

Table 6.9: Use case scenario UC-09: Cancel Donation (API only)

UC-10 — Release Pickup (API only)	
Primary actor	Recipient organisation (the accepting one). Secondary: administrator
Goal	Return a claimed pickup to the courier queue, for example when the courier cannot go.
Preconditions	The donation is <code>VOLUNTEER_ASSIGNED</code> and bound to the caller's organisation, which is still verified.
Trigger	A call to <code>POST /api/donations/{id}/status</code> with <code>status: ACCEPTED</code> . There is no portal button yet.
Main success flow	1. The server checks legality and role, then ownership, because acceptance from outside the open pool is a release. 2. It resolves the organisation, confirms verification, and recognises it is already bound. 3. It clears the courier, and does not change the acceptance counter or the frozen score. 4. The conditional write and <code>ACCEPTED</code> event are committed.
Alternative and exception flows	• Not the bound organisation: 404. • Already picked up: 409. • Organisation's verification revoked: 403.
Postconditions	The pickup is unclaimed again; couriers see only its coarse area.
Business rules	D-41: a release is not an acceptance. Known issue: an administrator release naming a <i>different</i> organisation re-binds the donation and counts a second acceptance.

Table 6.10: Use case scenario UC-10: Release Pickup (API only)

UC-11 — Manage Standing Requirements and Browse Needs Board	
Primary actor	Recipient organisation. Secondary: donor (reads the needs board)
Goal	Publish what the organisation needs so that donors see demand before supply exists.
Preconditions	Signed in as an organisation with a profile. Verification is not needed to post, but only verified organisations' needs reach donors.
Trigger	The organisation uses <i>Food Requirements & Demand Signals</i> ; a donor opens <i>Needs Board</i> .
Main success flow	1. The organisation posts a need (POST /api/requirements): food type, quantity and unit, beneficiaries, urgency, recurrence, notes. 2. It edits a need, or marks it fulfilled or no longer needed, which sets <code>isActive: false</code> (PATCH /api/requirements/{id}). 3. It views retired needs (<code>includeInactive=true</code>) and reopens one by setting <code>isActive: true</code>. 4. A donor's needs board lists the active needs of verified organisations (GET /api/requirements).
Alternative and exception flows	• Another organisation's requirement id: 404. • Non-positive quantity, negative beneficiaries or over-long text: 422, and nothing is written. • An explicit <code>null</code> for a field: the field is left unchanged. • Administrator without an organisation profile: 422. • Courier: an empty list.
Postconditions	The row is always kept (no deletion). An active need can break ranking ties and add a reason on the next read; frozen scores never change.
Business rules	D-29 (one-flag lifecycle), D-44 and D-46 (scopes), D-52 (ranking effect).

Table 6.11: Use case scenario UC-11: Manage Standing Requirements and Browse Needs Board

UC-12 — Edit Organisation Profile (verification-sensitive)	
Primary actor	Recipient organisation
Goal	Keep the organisation's details correct.
Preconditions	Signed in as an organisation with a profile.
Trigger	The organisation saves the <i>NGO / Recipient Profile</i> page.
Main success flow	1. The organisation edits name, type, address, capacity, contact person or phone and saves (PATCH /api/recipients/me). 2. The server computes the real changes. Omitted fields are ignored; <code>null</code> clears only nullable fields. 3. If the organisation is verified and its name, latitude or longitude actually changes, the server clears <code>is_verified</code> in the same commit. 4. It applies the changes and commits, and the profile shows the resulting verification state.
Alternative and exception flows	• Capacity not positive or coordinates out of range: 422. • Name resubmitted unchanged, or only type, address text, capacity or contact changed: verification is kept. • Coordinates can be set or cleared only through the API; clearing a pin also voids verification.
Postconditions	An organisation whose verification was voided is immediately excluded from ranking, the open pool and acceptance until re-verified (UC-13).
Business rules	D-54: verification vouches for identity and location, so changing either voids it. An organisation cannot set <code>is_verified</code> itself (D-16).

Table 6.12: Use case scenario UC-12: Edit Organisation Profile (verification-sensitive)

UC-13 — Verify or Revoke Organisation	
Primary actor	Administrator
Goal	Vouch for a genuine organisation, or withdraw that vouching.
Preconditions	Signed in as an administrator; the organisation exists.
Trigger	The administrator uses the verify control on <i>Partner Organizations Directory</i> .
Main success flow	1. The page lists all organisations with verification state, reliability and a count awaiting verification. 2. After checking the organisation outside the system, the administrator verifies it (POST /api/admin/recipients/{id}/verify). 3. The server sets <code>is_verified</code> and commits, and the list refreshes with a confirmation. 4. To revoke, the administrator withdraws verification (DELETE on the same route).
Alternative and exception flows	• Non-administrator: 403. • Unknown organisation: 404.
Postconditions	A verified organisation (with coordinates) is ranked, sees the open pool and can accept on its next request. A revoked one loses them on its next request; donations it already accepted are unaffected.
Business rules	Verification is a human judgement with no automated checks (D-37). Organisations created by an administrator start verified.

Table 6.13: Use case scenario UC-13: Verify or Revoke Organisation

UC-14 — Run Expiry Sweep	
Primary actor	Administrator
Goal	Record donations nobody accepted in time as losses.
Preconditions	Signed in as an administrator.
Trigger	The administrator runs the sweep from <i>Platform Donation Ledger</i> .
Main success flow	1. The client sends POST <code>/api/admin/maintenance/expire</code>. 2. The server selects AVAILABLE and MATCHED donations whose deadline is before now. 3. For each, it appends an EXPIRED event with no actor and the note “Deadline passed with no recipient”, and sets the status. 4. It commits and returns the number expired, which the page reports.
Alternative and exception flows	• Nothing overdue: the count is 0. • Non-administrator: 403.
Postconditions	Overdue donations are terminal and count towards the expiry-loss rate.
Business rules	Not scheduled. Overdue ACCEPTED donations are not touched. Its status write is not conditional (known issue).

Table 6.14: Use case scenario UC-14: Run Expiry Sweep

UC-15 — Manage User Accounts (API only)	
Primary actor	Administrator
Goal	Create accounts of any role, and suspend, restore, rename or re-role accounts.
Preconditions	Signed in as an administrator.
Trigger	Calls to <code>/api/admin/users</code> . There is no portal screen yet.
Main success flow	1. List accounts, optionally by role (GET). 2. Create an account of any role, including administrator (POST). Profile rows are created as at registration, and a created organisation starts verified. 3. Update <code>isActive</code>, <code>role</code>, <code>name</code>, <code>organization</code> or <code>phone</code> (PATCH <code>/api/admin/users/{id}</code>).
Alternative and exception flows	• Duplicate email: 409. • Suspending or demoting one's own administrator account: 409. • Suspending or demoting the last active administrator: 409. • Unknown user: 404.
Postconditions	Changes take effect on the affected user's next request: a suspended user receives 401, and 403 at sign-in.
Business rules	The first administrator comes only from <code>python -m foodlink.cli create-admin</code> (D-04).

Table 6.15: Use case scenario UC-15: Manage User Accounts (API only)

Table 6.16 summarises the remaining use cases in Figure 6.1, which are simple enough not to need a full scenario.

6.2 Class Diagram

Figure 6.2 shows the domain model. It contains the six persistent entities and two enumerations from `models.py`, and the `matching` module with the two data classes it produces.

Entities and attributes. Each class lists its mapped columns with their types. Multiplicity `[0..1]` marks a nullable column, `{PK}` and `{FK}` mark keys, and a leading slash marks a derived, computed property: `User.initials`, the profile ids, and `Recipient.reliability_score`. `Donation.timestamp_of(status)` returns when a donation first entered a state by scanning its events.

Associations and multiplicities. Multiplicities follow the foreign keys exactly. A `User` may act for at most one `Recipient`, and a recipient may exist without a user,

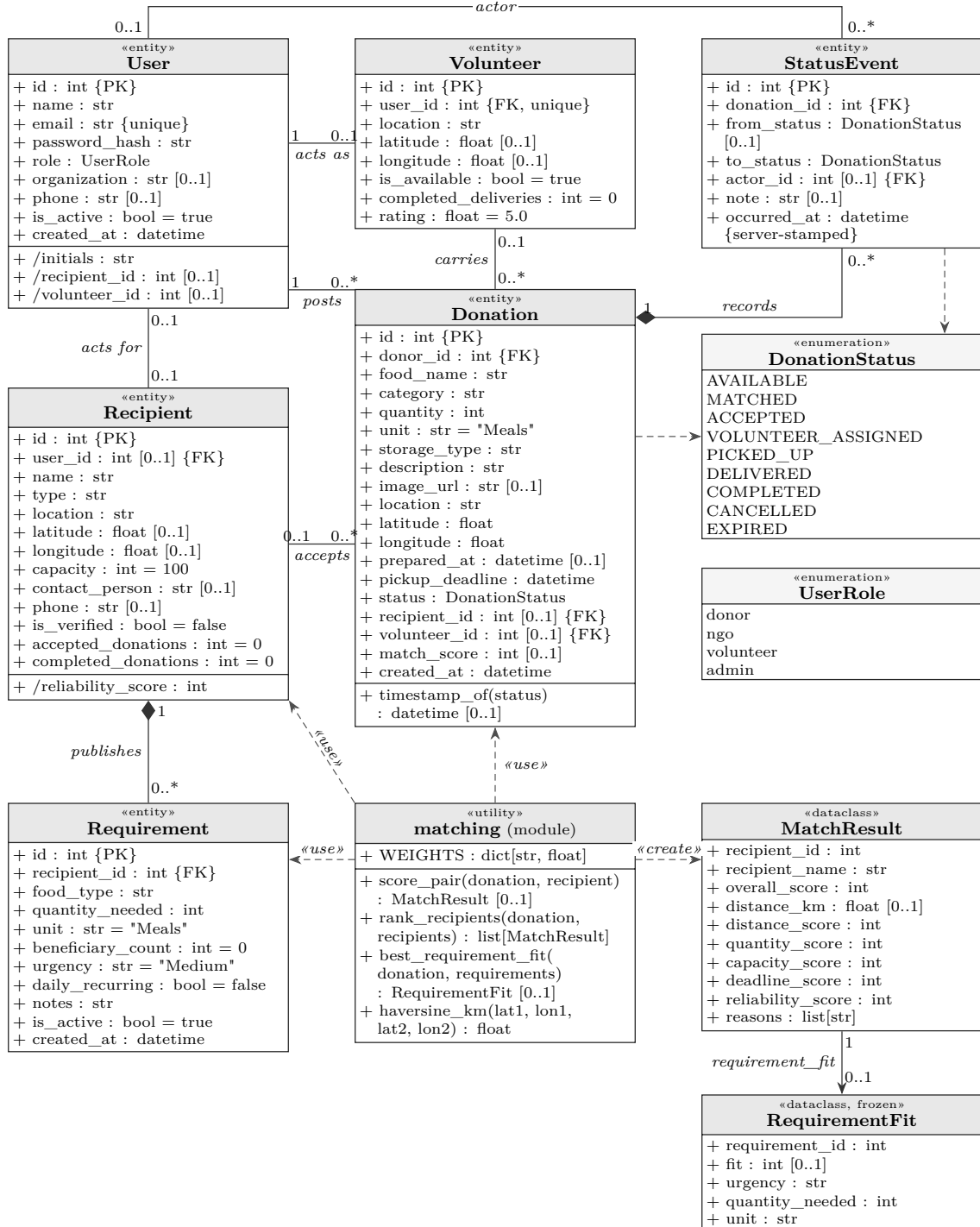


Figure 6.2: Class diagram of the FoodLink domain model and matching module

Use case	Actor	Behaviour
Manage Own Account	Any signed-in role	Edit name, organisation and phone (PATCH /api/auth/me); change password, proving the current one (POST /api/auth/password).
Track Donation Status	Any signed-in role	Read the donations one's scope allows, each with its event timeline (GET /api/donations).
Browse Unclaimed Pickups	Courier	The courier queue within UC-06, showing a coarse pickup area only.
Set Courier Availability	Courier	Toggle on or off duty and set a base location (PATCH /api/volunteers/me). Availability is advisory.
View Platform Metrics	Administrator	Analytics screens read GET /api/metrics (the API allows any signed-in role).
Override Lifecycle Transition	Administrator	Perform any transition in Table 2.10 on a party's behalf, under the same legality rules; the claim is excluded.

Table 6.16: Summary of the remaining use cases

because `recipients.user_id` is nullable (seeded organisations). A **User** may act as at most one **Volunteer**, and every volunteer has exactly one user, because the key is unique and mandatory. A user posts any number of donations. A **Donation** is accepted by *zero or one* recipient and carried by *zero or one* volunteer, since both keys stay null until acceptance and claim. A **StatusEvent** optionally names the user who acted; the expiry sweep acts with no user.

Composition. **Donation–StatusEvent** and **Recipient–Requirement** are compositions (filled diamonds). The parts belong to exactly one whole, and the ORM relationships are declared with cascading deletion. Events are append-only in practice: the application never updates or deletes them.

The matching module. `matching` is shown as a `«utility»` because it is a Python module of functions, not an instantiable class. It uses **Donation**, **Recipient** and **Requirement** rows passed in by the routers, and creates transient **MatchResult** and **RequirementFit** objects that are never persisted. Only the integer `overall_score` survives, copied into `Donation.match_score` at posting and at first acceptance.

What is deliberately absent. There is no inheritance in the domain. Roles are an enumerated attribute on one `users` table, and **Recipient** and **Volunteer**

are profiles associated with an account, not subclasses. Pydantic schemas, routers and framework classes are implementation infrastructure and are omitted to keep the diagram readable. The dependency of `User.role` on `UserRole` is shown by its attribute type alone, because an edge would have to cross the diagram.

6.3 Activity Diagram

Figure 6.3 models the core surplus-food redistribution workflow. It runs from posting, through matching, acceptance, courier claim, pickup and delivery, to confirmation, together with the alternative paths the server implements.

Partitions. The five swimlanes are the four roles and the FoodLink server. Every action that changes stored state is placed in the server lane, because only the server records transitions. The human lanes contain the decisions and physical actions that trigger them.

Main flow. The donor submits a donation. After validation, the server stores it, ranks eligible organisations and, if any exist, records `MATCHED`; otherwise the donation stays `AVAILABLE`. Both branches merge into the open pool. A verified organisation accepts. The server’s decision node checks, in effect, verification, the deadline and whether the conditional write still finds the donation open. On success it binds the organisation and records `ACCEPTED`. A merge node in the courier lane leads to the unclaimed queue, where a courier claims the pickup. A second decision reflects the conditional claim. The courier then collects and delivers the food, each step recorded by the server, and the organisation confirms receipt, which updates the completion counters and records `COMPLETED`.

Alternative paths. Flow-final nodes (circles with a cross) mark requests the server refuses, which leave the donation unchanged. If no organisation accepts before the deadline, the administrator’s expiry sweep ends the flow in `EXPIRED`. The *release* loop returns a claimed pickup to the courier queue; it is available through the API only. The dashed *interruptible region* spans the states in which the donor may still cancel, from `AVAILABLE` to `VOLUNTEER_ASSIGNED`. The zig-zag interrupting edge from *Cancel request* leaves the region and ends in `CANCELLED`, withdrawing any counted acceptance.

Simplifications. To keep the diagram readable, three administrator-only paths described in Table 2.10 are not drawn: cancellation after pickup, `ACCEPTED` → `EXPIRED` through the status route, and administrators performing other parties’ transitions.

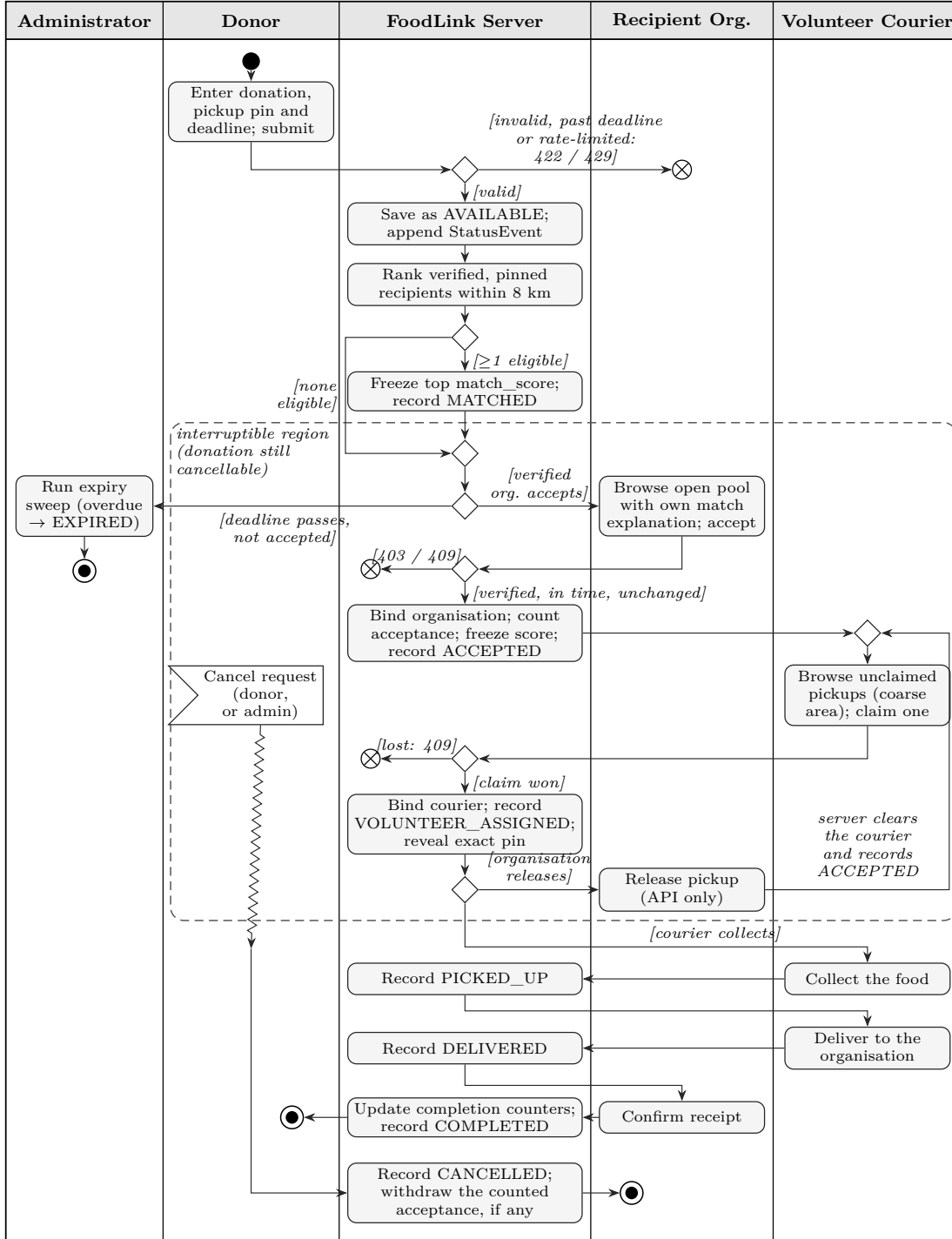


Figure 6.3: Activity diagram of the donation lifecycle, from posting to completion, expiry or cancellation. Codes on refusal edges: 422 invalid input or past deadline; 429 rate limit; 403 unverified organisation; 409 overdue, already taken or claim lost.

The diagram shows one courier and one organisation. Concurrent competitors appear only through the refusal edges they cause.

Bibliography

- [BRJ05] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. 2nd ed. Boston: Addison-Wesley, 2005 (cit. on p. 63).
- [Foo26a] FoodLink Project Team. *FoodLink Architecture, Decision and Task Records*. `ai/ARCHITECTURE.md`, `ai/DECISIONS.md` (D-01 to D-61), `ai/TASKS.md` and `ai/PROJECT_STATE.md` in the FoodLink repository. 2026 (cit. on pp. 10, 17).
- [Foo26b] FoodLink Project Team. *FoodLink Source Repository*. Git repository, branch `master` at commit `e588b35`; backend in `code/foodlink/`, frontend in `frontend/src/`, tests in `code/tests/`. 2026 (cit. on p. 10).
- [GSHS26] Rishi Garg, Aayushi Sud, Yugam Heer, and Himanshi Singla. *FoodLink: A Coordination Platform for Surplus Food Redistribution — Project Proposal for UCS503P*. Project proposal, `project-proposal/main.tex` in the FoodLink repository. 2026 (cit. on pp. 9, 11, 13, 17, 44–46, 52, 53, 56).
- [Har12] Dick Hardt. *The OAuth 2.0 Authorization Framework*. Request for Comments 6749. Internet Engineering Task Force, 2012 (cit. on p. 36).
- [JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Request for Comments 7519. Internet Engineering Task Force, 2015 (cit. on p. 36).
- [Kle17] Martin Kleppmann. *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017 (cit. on p. 31).
- [Met26] Meta Open Source. *React Documentation*. 2026. URL: <https://react.dev/> (cit. on pp. 19, 49).
- [Min23] Ministry of Law and Justice, Government of India. *The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023)*. The Gazette of India, Extraordinary, Part II, Section 1. 2023 (cit. on p. 55).
- [Obj17] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. formal/17-12-05. Object Management Group. 2017 (cit. on p. 63).
- [OWA21] OWASP Foundation. *OWASP Top 10:2021*. 2021. URL: <https://owasp.org/Top10/> (cit. on p. 41).
- [PM15] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th ed. New York: McGraw-Hill Education, 2015 (cit. on pp. 43, 49).

- [PM99] Niels Provos and David Mazières. “A Future-Adaptable Password Scheme”. In: *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*. 1999, pp. 81–91 (cit. on p. 36).
- [Pyd26] Pydantic Services. *Pydantic Documentation*. 2026. URL: <https://docs.pydantic.dev/> (cit. on pp. 19, 49).
- [pyt26] pytest-dev. *pytest Documentation*. 2026. URL: <https://docs.pytest.org/> (cit. on pp. 19, 49).
- [Ram26] Sebastián Ramírez. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/> (cit. on pp. 19, 49).
- [Sin84] Roger W. Sinnott. “Virtues of the Haversine”. In: *Sky and Telescope* 68.2 (1984), p. 159 (cit. on p. 33).
- [Som16] Ian Sommerville. *Software Engineering*. 10th ed. Boston: Pearson, 2016 (cit. on pp. 17, 49).
- [SQL26a] SQLAlchemy Authors. *Alembic Documentation*. 2026. URL: <https://alembic.sqlalchemy.org/> (cit. on pp. 19, 49).
- [SQL26b] SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation*. 2026. URL: <https://docs.sqlalchemy.org/en/20/> (cit. on pp. 19, 49).
- [SQL26c] SQLite Consortium. *Appropriate Uses For SQLite*. 2026. URL: <https://www.sqlite.org/whentouse.html> (cit. on pp. 19, 50).
- [Tha26] Thapar Institute of Engineering and Technology. *TIET Report Template (tietreport.cls v1.0.4)*. Overleaf. 2026. URL: <https://www.overleaf.com/read/rjgcdqyqkxnd> (cit. on p. 14).
- [Uni24] United Nations Environment Programme. *Food Waste Index Report 2024*. Technical report. Nairobi: United Nations Environment Programme, 2024 (cit. on p. 9).
- [Ven26] Raghav B. Venkataramaiyer. *UCS503 Semester Project: Deliverables and Evaluation Timeline*. Thapar Institute of Engineering and Technology. 2026. URL: <https://tiet-ucs503.github.io/semester-project/> (cit. on pp. 9, 54).
- [Vit26a] Vite Contributors. *Vite Documentation*. 2026. URL: <https://vite.dev/> (cit. on pp. 19, 49).
- [Vit26b] Vitest Contributors. *Vitest Documentation*. 2026. URL: <https://vitest.dev/> (cit. on pp. 19, 49).